NAPQES: A Single-Primitive Authenticated Encryption Scheme Built from HMAC-SHA256

Abdeslam Jakjoud[1] and Karim Amor[1]

[1]QAegis, France

May 2026
Preprint — submitted to IACR ePrint

Abstract

We present NAPQES, an Authenticated Encryption with Associated Data (AEAD) scheme whose security reduces exclusively to the pseudorandomness of HMAC-SHA256. Unlike AES-GCM and ChaCha20-Poly1305, NAPQES does not employ finite-field arithmetic, group operations, or AES S-box permutations, and is therefore not subject to Shor's algorithm or hidden-subgroup quantum attacks. The construction combines a prime-indexed token cipher, HMAC-derived noise injection to achieve ciphertext pseudorandomness, HMAC-derived plaintext padding for cross-implementation determinism, and a standard HMAC-SHA256 authentication tag. We describe the wire format (version 6, frozen), prove IND-CPA security under the PRF assumption on HMAC-SHA256, discuss the post-quantum security level, and provide performance measurements and implementation notes. The streaming Release of Unverified Plaintext (RUP) caveat present in earlier versions is addressed by a new online-AE streaming format (`encrypt_stream_ae`/`decrypt_stream_ae`) that verifies a per-chunk HMAC-SHA256 tag before releasing any plaintext to the caller. A reference implementation in Python and Rust, together with a 30-vector Known-Answer Test corpus and a NIST SP 800-22 Rev 1a statistical analysis, are published at `https://github.com/[REPO]`.

Contents

1 Introduction

Modern AEAD standards—AES-GCM [1] and ChaCha20-Poly1305 [2]—have achieved widespread deployment. Their security, however, ultimately depends on algebraic structures (finite-field multiplication for GCM, add-rotate-XOR chains for ChaCha20) whose resilience against quantum adversaries running variants of Shor's algorithm is an active research question [7].

NAPQES takes a more conservative approach: the *only* cryptographic primitive in the construction is HMAC-SHA256, approved under NIST FIPS 198-1 and FIPS 180-4. The security of the entire scheme therefore reduces to the pseudorandomness of HMAC-SHA256—a well-studied assumption with no known quantum speedup beyond Grover's $O(\sqrt{N})$ search, which reduces the effective security level from 256 bits to approximately 128 bits while remaining far above all recommended thresholds.

Contributions.

1. We specify NAPQES wire format v6 (Section 3), including the key serialisation, HMAC derivation schedule, token construction, padding scheme, and authentication tag.
2. We prove IND-CPA security (Section 4) under the PRF assumption on HMAC-SHA256, with an explicit key-entropy term accounting for the fact that the HMAC key is a low-entropy, structured prime tuple rather than a uniform bit-string (Theorem 4.7, Section 4.3), and show that the authentication tag provides EUF-CMA integrity.
3. We provide a post-quantum analysis (Section 5) and a comparison with AES-GCM and ChaCha20-Poly1305 (Section 6).
4. We report NIST SP 800-22 Rev 1a statistical test results on 10 million bits of NAPQES ciphertext output (Section 8), a 30-vector KAT corpus (Section 9), and a fuzz harness covering the full decoder pipeline (Section 10).
5. We discuss known caveats (Section 11): the 16-bit plaintext length cap (CAV-002), the padding length-bucket leakage (CAV-003), and the ciphertext expansion bound (CAV-004). The streaming Release of Unverified Plaintext issue (CAV-001) has been resolved by the online-AE streaming format introduced in this version.
6. We state the normative status of, and selection rule among, the three wire encodings — block-mode v7, the deprecated basic streaming format, and the online-AE streaming format (Section 3.9) — closing audit finding CVF7.

2 Related Work

AES-GCM [1]. AES-GCM combines AES-CTR for confidentiality with a GHASH polynomial MAC over GF(2^{128}). The Forbidden Attack [8] and nonce-reuse attacks [9] demonstrate that the algebraic structure of GHASH can be exploited when the polynomial MAC key is recovered (e.g., under nonce misuse), algebraically recovering the authentication key. NAPQES uses HMAC-SHA256 for authentication, which does not admit the Forbidden Attack.

ChaCha20-Poly1305 [2]. ChaCha20 is an ARX cipher whose security is not reduced to a well-studied hardness assumption; Poly1305 is again a polynomial MAC, subject to the same class of Forbidden Attacks as GHASH when the one-time key is reused. Neither ChaCha20 nor Poly1305 is approved under current FIPS standards.

Ascon [3]. Ascon (NIST LWC winner) is a sponge-based AEAD targeting lightweight environments. Its security is based on the hardness of distinguishing the Ascon permutation from a random permutation, which involves xor-and-rotate operations over 64-bit words. Ascon is not yet in a FIPS-approved family.

HMAC-based constructions. HKDF [10] and HMAC-DRBG [11] demonstrate that HMAC-SHA256 is a sufficient basis for key derivation and pseudorandom generation. NAPQES extends this philosophy to the full AEAD context.

3 Construction

3.1 Notation

Throughout this paper:

- $\|$ denotes byte concatenation.
- $\mathrm{HMAC}(K, M)$ denotes HMAC-SHA256 keyed with K over message M.
- $\mathrm{be5}(x)$ denotes the 5 least-significant bytes of integer x, big-endian encoded.
- $\mathrm{be8}(x)$ denotes the 8-byte big-endian encoding of integer x (the fixed-width token field introduced by the CVF1 fix; see Section 3 "Wire Format (Version 7)").
- $\mathrm{varint}(x)$ denotes unsigned LEB128 encoding of non-negative integer x, defined precisely in Definition 3.2 below. **Retired for block-mode tokens as of the CVF1 fix** (Section 3); still used by the streaming format and by legacy v6 ciphertext decoding.
- $\mathcal{P}$ denotes the set of prime integers in $[10^6, 9.9 \times 10^6]$.

Remark 3.1 (Audit finding CVF17: LEB128 used without a definition or normative reference)**.** *The notation above previously stated only that* $\mathrm{varint}(x)$ *is "unsigned LEB128 encoding of non-negative integer* x*," without ever defining LEB128 or citing a normative reference for it. As written, the byte layout, the continuation-bit convention, and whether a canonical (minimal-length) encoding is required were all left implicit, so the wire format could not be reproduced from the specification alone. Beyond being undefined, variable-length LEB128 decoding is error-prone in ways that have security relevance for a decoder (overlong/non-canonical encodings, continuation-bit handling, unbounded length); those concrete decoding hazards are tracked as a separate implementation-level finding and are not the subject of this remark. Definition 3.2 below states the encoding explicitly and specifies canonicality and rejection rules for decoding.*

Definition 3.2 (Unsigned LEB128 encoding)**.** *For a non-negative integer* x*, write* x *in base* 2^7 *as* $x = \sum_{i=0}^{\ell-1} g_i \cdot 2^{7i}$ *with each* group $g_i \in [0, 127]$*, where* ℓ *is chosen minimally (i.e.* $g_{\ell-1} \neq 0$*, except* $\ell = 1$ *and* $g_0 = 0$ *when* $x = 0$*). The encoding* $\mathrm{varint}(x)$ *is the* ℓ*-byte sequence* $b_0 \| b_1 \| \cdots \| b_{\ell-1}$ *(*little-endian group order*: the least-significant 7-bit group is emitted first) where*

$$b_i = g_i \ \vee \ (\texttt{0x80} \text{ if } i < \ell - 1 \text{ else } \texttt{0x00}),$$

i.e. each byte carries 7 data bits in its low-order bits, and the high-order (8th) bit is a continuation flag*: set to* 1 *if another group follows and* 0 *on the final byte. Decoding reverses this: read bytes, accumulating* $\mathrm{group}(b_i) = b_i \,\&\, \texttt{0x7F}$ *left-shifted by* $7i$ *into the result, until a byte with the continuation flag clear is read.*

Canonicality. *A conforming decoder accepts* only *the canonical, minimal-length encoding: the unique* ℓ *satisfying* $2^{7(\ell-1)} \leq x < 2^{7\ell}$ *for* $x > 0$ *(equivalently,* $b_{\ell-1} \neq \texttt{0x00}$*), or the single byte* `0x00` *for* $x = 0$*. Decoding* ***MUST reject****:*

1. Non-canonical (overlong) encodings — *any encoding using more bytes than the minimal ℓ above, including a trailing group $g_{\ell-1} = 0$ appended after what would otherwise be the final byte;*

2. Inputs exceeding the maximum permitted integer width — *this specification bounds every* varint*-encoded value to $x < 2^{64}$ (consistent with the* be8 *fixed-width field it replaces for block-mode tokens, Section 3.7), so a conforming decoder rejects any input requiring more than $\lceil 64/7 \rceil = 10$ groups, and truncated input (a continuation-flagged byte with no following byte).*

These rules make the mapping between integers and their encodings a bijection on $[0, 2^{64})$, eliminating the ambiguity (multiple valid encodings of the same integer) that overlong LEB128 encodings would otherwise introduce.

3.2 Key

A NAPQES key $\mathbf{k} = (k_1, k_2, \ldots, k_K)$ is an ordered tuple of K distinct primes drawn uniformly from $\mathcal{P}$ using a cryptographically secure DRBG. The serialised key material is:

$$\mathsf{key\_bytes} = \mathrm{be5}(k_1) \| \mathrm{be5}(k_2) \| \cdots \| \mathrm{be5}(k_K).$$

The cardinality of $\mathcal{P}$ is approximately 586 000; a 10-element ordered tuple provides a key space of:

$$\frac{|\mathcal{P}|!}{(|\mathcal{P}| - 10)!} \approx 1.1 \times 10^{58} \approx 2^{196}.$$

The relevant security quantity for Theorem 4.7 below is not the raw bit-length $8 \cdot 5K = 40K$ of key_bytes, but the *min-entropy* of $\mathbf{k}$ as sampled by KeyGen:

$$H_\infty(\mathbf{k}) \;=\; \log_2\!\left(\frac{|\mathcal{P}|!}{(|\mathcal{P}| - K)!}\right) \;\approx\; K \log_2 |\mathcal{P}|,$$

i.e. exactly $\log_2$ of the ordered-tuple key-space size above (≈ 196 bits for $K = 10$, ≈ 19 bits for $K = 1$). Because kb is a deterministic, injective encoding of $\mathbf{k}$, $H_\infty(\mathsf{kb}) = H_\infty(\mathbf{k})$: injective post-processing of a random variable never increases its min-entropy above the source's, and here it does not decrease it either (Remark 4.17 discusses the one place this is not quite free — HMAC's own key-length handling for $5K > 64$ bytes). This distinction — $H_\infty(\mathbf{k}) \ll 40K$ — is the subject of audit finding **CVF8** below.

3.3 Domain-Separated HMAC Derivation

Remark 3.3 (CVF2 fix: unified domain-first layout)**.** *An earlier revision of this construction used a* nonce-first *layout $N \| d \| \mathsf{ctx}$ for domains $\{0, 1, 2, 4, 5, 6, 7\}$ but a* domain-first*, AAD-binding layout for the authentication-tag and streaming-AE domains ($\{3, 8, 9\}$). Because Table 1 and this section stated only the nonce-first formula, reading the wire-format tag definition (Section 3.7) through that formula silently dropped the AAD from the modelled HMAC input, and Remark 4.1 had to reason about the two layouts as separate groups, producing a probabilistic cross-group collision term instead of an unconditional separation argument. This was audit finding **CVF2**. The fix below unifies all ten domains onto a single domain-first layout, after which Remark 4.1 needs only a single, unconditional injectivity argument.*

All internal values are computed as:

$$\mathrm{Derive}_d(\mathsf{kb}, N, \mathsf{ctx}) = \mathrm{HMAC}(\mathsf{kb},\ d \| N \| \mathsf{ctx}),$$

where $d \in \{0, 1, \ldots, 9\}$ is a 1-byte domain separator and every variable-width field inside ctx is length-prefixed (e.g. $\mathrm{be4}(|A|) \| A$ for AAD A).

Domain	Function	Context	Output
0x00	Noise position	be5(ct_pos)	Boolean
0x01	Real-token addend	be5($real\_idx$)	$[1, k_i - 1]$
0x02	Noise probability	(none)	$[0.75, 0.99]$
0x03	Auth tag (block)	be4($\|aad\|$)$\|aad\|\widetilde{B}$	32 bytes
0x04	Noise character	be5(ct_pos)	$[32, 127]$
0x05	Noise-token addend	be5(ct_pos)	$[1, k_i - 1]$
0x06	Padding codepoint	be4(pad_idx)	$[32, 126]$
0x07	Varint keystream	be4($block\_idx$)	32-byte block
0x08	Chunk tag (stream AE)	be4($\|aad\|$)$\|aad\|$be4(i)$\|\tilde{C}_i$	32 bytes
0x09	Final sentinel tag (stream AE)	be4($\|aad\|$)$\|aad\|$be4(n_{chunks})	32 bytes

Table 1: Domain-separated HMAC derivation schedule (CVF2 fix: unified domain-first layout, $\text{Derive}_d(\mathsf{kb}, N, \mathsf{ctx}) = \text{HMAC}(\mathsf{kb}, d\|N\|\mathsf{ctx})$ for every domain; the nonce N is applied uniformly by Derive_d and so is omitted from the Context column above). Domains 0x08–0x09 are used exclusively by the online-AE streaming format (Section 3.8).

3.4 Noise Probability

$$t = \frac{\text{Derive}_{0x02}(\mathsf{kb}, N, \varepsilon)[0:8]_{\text{be64}}}{2^{64}}, \quad p = 0.75 + t \cdot 0.24.$$

3.5 Plaintext Padding

Let $n = |P|$ (plaintext length in codepoints). Define:

$$B = \max(16,\ 2^{\lceil \log_2(n+1) \rceil}), \qquad P_d = [n \gg 8,\ n \wedge 0\text{FF}]\|P\|\mathbf{pad},$$

where $\mathbf{pad} = (\mathbf{pad}_0, \ldots, \mathbf{pad}_{B-n-1})$ is a $(B-n)$-codepoint sequence, each codepoint HMAC-derived via domain 0x06 as made precise by Definition 3.5 below. The total padded length is $n + 2 + (B - n) = B + 2$ codepoints.

Remark 3.4 (Audit finding CVF18: the padding-codepoint derivation formula was missing)**.** *The text above previously named domain 0x06 and its output range* $[32, 126]$ *(Table 1) but never stated how the 32-byte output of* $\text{Derive}_{0x06}(\mathsf{kb}, N, \text{be4}(pad\_idx))$ *is reduced to a single codepoint in* $[32, 126]$ *— which bytes are taken and what mapping is applied. Without that formula the padding is not reproducible from the specification alone, and an unstated reduction into a 95-value range is exactly the kind of construction that can introduce modulo bias if done naively. Definition 3.5 below states the formula explicitly, matching the reference implementation (napqes.py's _pad_message), and Remark 3.11 is extended to cover its bias.*

Definition 3.5 (Padding codepoint — domain 0x06)**.** *Let* $w = \text{Derive}_{0x06}(\mathsf{kb}, N, \text{be4}(pad\_idx))[0{:}4]_{\text{be32}} \in [0, 2^{32})$*, i.e. the first 4 bytes of the 32-byte HMAC output, interpreted as an unsigned big-endian integer (the same* $[0{:}n]_{\text{be}k}$ *convention used by Definitions 3.8–3.10). Then*

$$\mathbf{pad}_{pad\_idx} = (w \bmod 95) + 32 \ \in [32, 126].$$

3.6 Token Construction

Remark 3.6 (Audit finding CVF6: token-emission pseudocode's helper primitives were undefined)**.** *The token-emission loop below invokes four helper primitives — is_noise (domain 0x00), derive_noise_char (0x04), derive_noise_addend (0x05), and derive_real_addend (0x01) — that were previously named only by Table 1's context/range columns, with no formula stating how the 32-byte* Derive_d *output is reduced to the required Boolean decision or bounded integer. As written, the pseudocode could not be executed and the token construction was not*

*reproducible from the paper alone — and the exact reduction (and hence any modulo bias) was unspecified for every one of these primitives. This was audit finding **CVF6**. Definitions 3.7–3.10 below give each primitive as an explicit, byte-exact reduction of* $\mathrm{Derive}_d(\mathsf{kb}, N, \mathsf{ctx})$*, matching the reference implementations exactly, and Remark 3.11 bounds the resulting modulo bias for each. This is a specification-completeness fix only — no algorithm or wire-format change; the reference implementations already implemented exactly the formulas given below.*

For each padded codepoint c (with current *ct_pos* and *real_idx*), the token is emitted by the loop below, which invokes the four helper primitives `is_noise`, `derive_noise_char`, `derive_noise_addend`, and `derive_real_addend`. Definitions 3.7–3.10 give each primitive as an explicit reduction of $\mathrm{Derive}_d(\mathsf{kb}, N, \mathsf{ctx})$ (Table 1) into its target range, so that the loop is fully specified and reproducible byte-for-byte.

Throughout, $[0{:}n]_{\mathrm{be}k}$ denotes the first n bytes of a 32-byte HMAC-SHA256 digest, interpreted as an unsigned big-endian integer in $[0, 2^k)$ where $k = 8n$ (e.g. $[0{:}4]_{\mathrm{be}32} \in [0, 2^{32})$, matching the $[0{:}8]_{\mathrm{be}64}$ convention already used for the noise probability t above).

Definition 3.7 (`is_noise` — domain `0x00`)**.** *Let* $u = \mathrm{Derive}_{0x00}(\mathsf{kb}, N, \mathrm{be5}(ct\_pos))[0{:}8]_{\mathrm{be}64}/2^{64} \in [0, 1)$ *and let* p *be the noise probability of the previous subsection. Then*

$$\mathit{is\_noise}(ct\_pos) = \mathbf{1}[\, u < p \,].$$

This is a strict-inequality threshold test on a 64-bit fraction, not a modular reduction, so it carries no modulo bias; the only deviation from a continuous uniform draw is the 2^{-64} *discretisation granularity, which is cryptographically negligible.*

Definition 3.8 (`derive_noise_char` — domain `0x04`)**.** *Let* $w = \mathrm{Derive}_{0x04}(\mathsf{kb}, N, \mathrm{be5}(ct\_pos))[0{:}4]_{\mathrm{be}32} \in [0, 2^{32})$*. Then*

$$\mathit{derive\_noise\_char}(ct\_pos) = (w \bmod 96) + 32 \ \in [32, 127].$$

Definition 3.9 (`derive_noise_addend` — domain `0x05`)**.** *Let* $v = \mathrm{Derive}_{0x05}(\mathsf{kb}, N, \mathrm{be5}(ct\_pos))[0{:}4]_{\mathrm{be}32} \in [0, 2^{32})$*. Then, for key element* k*,*

$$\mathit{derive\_noise\_addend}(ct\_pos, k) = (v \bmod (k-1)) + 1 \ \in [1, k-1].$$

Definition 3.10 (`derive_real_addend` — domain `0x01`)**.** *Let* $v = \mathrm{Derive}_{0x01}(\mathsf{kb}, N, \mathrm{be5}(real\_idx))[0{:}4]_{\mathrm{be}32} \in [0, 2^{32})$*. Then, for key element* k*,*

$$\mathit{derive\_real\_addend}(real\_idx, k) = (v \bmod (k-1)) + 1 \ \in [1, k-1].$$

Remark 3.11 (Modulo bias of the finite-range reductions)**.** *Definitions 3.5, 3.8–3.10 reduce a value* v *drawn (pseudo-)uniformly from* $[0, 2^{32})$ *into* $[0, m)$ *via* $v \bmod m$ *before shifting into the target range. For any modulus* $m \le 2^{32}$*, every residue class occurs either* $\lfloor 2^{32}/m \rfloor$ *or* $\lceil 2^{32}/m \rceil$ *times among the* 2^{32} *possible values of* v*, so each residue's probability deviates from the ideal* $1/m$ *by at most* 2^{-32} *in absolute terms, i.e. by a factor of at most* $m/2^{32}$ *relative to* $1/m$*. For* ***derive_noise_char*** *(*$m = 96$*), this bound gives a relative deviation below* $3 \times 2^{-26} < 2^{-24}$*. For the padding codepoint (Definition 3.5,* $m = 95$*; audit finding CVF18, Remark 3.4), the same bound gives a relative deviation below* $95/2^{32} < 2^{-25}$*. For* ***derive_noise_addend*** *and* ***derive_real_addend****,* $m = k - 1$ *with* $k \in \mathcal{P} \subset [10^6, 9.9 \times 10^6] \subset [0, 2^{24})$*, giving a relative deviation below* $2^{24}/2^{32} = 2^{-8}$*. In all cases the bias is statistically dominated by, and computationally no more exploitable than, the underlying PRF advantage against HMAC-SHA256, and is therefore negligible for the security claims of Section 4; no rejection sampling is required.*

Listing 1: Token emission loop (pseudocode)

```
while is_noise(ct_pos):          # Definition 1, domain 0x00
    k        = key[real_idx % K]
```

```
    noise_c  = derive_noise_char(ct_pos)      # Definition 2, domain 0
       x04; in [32,127]
    noise_a  = derive_noise_addend(ct_pos, k) # Definition 3, domain 0
       x05; in [1,k-1]
    emit  noise_c * k + noise_a
    ct_pos += 1

k = key[real_idx % K]
a = derive_real_addend(real_idx, k)           # Definition 4, domain 0
   x01; in [1,k-1]
emit  c * k + a
ct_pos += 1;  real_idx += 1
```

Remark 3.12 (Retracted: the GCD claim below was vacuous (audit finding CVF4))**.** *An earlier revision claimed that, because k is prime and $a \in [1, k-1]$, every real token satisfies* $\gcd(\mathit{token}, k) < k$*, and that the same holds for noise tokens, making real and noise tokens "computationally indistinguishable under GCD analysis without the key." This is vacuous: since k is prime and every addend $a \in [1, k-1]$ by construction (Table 1, domains* `0x01/0x05`*),* $\gcd(a, k) = 1$ *unconditionally, and* $\gcd(\mathit{token}, k) = \gcd(c \cdot k + a,\ k) = \gcd(a, k) = 1$ *for* every *token — real or noise, with or without the key. The statement requires no HMAC output and no noise-schedule design to hold, so it establishes no indistinguishability property and is removed. The layer's genuine, distinct contribution — length-decorrelation from plaintext content, not GCD-level structural indistinguishability — is stated and proved in Lemma 4.15 below.*

3.7 Wire Format (Version 7 — CVF1 fix)

Remark 3.13 (Audit finding CVF1 and the fix below)**.** *Version 6 of this construction encoded each token with* varint$(\cdot)$ *(LEB128). Because* token $= c \cdot k + a$*, the LEB128 byte-length of a token grows with the plaintext codepoint c: a codepoint near 1 yields a $\approx 2^{20}$ token (≈ 3 LEB128 bytes), while a codepoint near $2^{16} - 1$ yields a $\approx 2^{36}$ token (≈ 6 LEB128 bytes). Two plaintexts with equal padded codepoint count but different codepoint values therefore produced token blobs B_0, B_1 of* different byte-length *with overwhelming probability, so the masked ciphertext $\widetilde{B}$ leaked plaintext content through its length alone — an IND-CPA distinguishing advantage ≈ 1. This invalidated the "B is the same length for both plaintexts" step of the hiding lemma (Lemma 4.12 below). The fix, described here, is to replace* varint$(\cdot)$ *with a fixed-width encoding so that $|B|$ is a function only of the token* count*, which in turn is a function of the padded codepoint count and the HMAC-derived, content-independent noise schedule (domain* `0x00`*) — never of codepoint values. Lemma 4.12 is restated and re-proved below under this v7 encoding; Section 4.13 discusses the v6 argument's error precisely.*

Each token is serialised as be8(token) instead of varint(token):

$$B = \mathrm{be8}(\mathrm{token}[0]) \| \mathrm{be8}(\mathrm{token}[1]) \| \cdots \| \mathrm{be8}(\mathrm{token}[N-1]),$$

so $|B| = 8N$ where N (real + noise tokens) depends only on the padded codepoint count and $(\mathsf{kb}, N_{\text{once}})$ via the noise-position oracle (domain `0x00`) — not on codepoint values. The overall ciphertext is unchanged in shape:

$$C = N \| \widetilde{B} \| T,$$

where N is a 16-byte random nonce and $\widetilde{B}$ is the domain-`0x07` XOR-masked fixed-width token blob, defined explicitly as

$$\widetilde{B} \;=\; B \oplus \mathrm{ks}_{0x07}(N)[0 : |B|], \qquad \mathrm{ks}_{0x07}(N) \;=\; \mathrm{Derive}_{0x07}(\mathsf{kb}, N, \mathrm{be4}(0)) \,\|\, \mathrm{Derive}_{0x07}(\mathsf{kb}, N, \mathrm{be4}(1)) \,\|\, \cdots,$$

i.e. the encode-then-mask order is: (1) assemble the fixed-width token blob B from the token vector $(\mathrm{token}[0], \ldots, \mathrm{token}[N-1])$ produced by the token-construction loop above (Definitions 3.7–3.10);

(2) generate the domain-`0x07` keystream $\mathrm{ks}_{0x07}(N)$ as the concatenation of 32-byte Derive_{0x07} blocks indexed $0, 1, 2, \ldots$ (Table 1), generating only as many blocks as needed to cover $|B|$ bytes and truncating the final block to the remaining length; (3) XOR the two byte strings of equal length $|B|$. This equation is proved to yield a value statistically independent of, and (for fixed N) uniformly distributed given, B in the "Independence of $\mathrm{ks}(N^*)$ from B" argument of Lemma 4.12's proof, which this definition forward-references.

Remark 3.14 (Audit finding CVF19: $\widetilde{B}$ used without a defining equation at its point of introduction)**.** *This subsection previously introduced $\widetilde{B}$ only in prose — "the domain-`0x07` XOR-masked fixed-width token blob" — with no defining equation at this point; the relation $\widetilde{B} = B \oplus \mathrm{ks}_{0x07}(N)$, the encode-then-mask operation order, and the keystream length/truncation rule were stated only later, inside Lemma 4.12's proof (Section 4.13 and surrounding text). As a result, $C = N\|\widetilde{B}\|T$ could not be reproduced from this section alone. The equation above closes that gap by stating the definition explicitly here, with forward references to the token construction (Definitions 3.7–3.10) and the domain-`0x07` keystream construction it depends on.*

$$T = \mathrm{Derive}_{0x03}(\mathtt{kb},\ N,\ \mathrm{be4}(|A|)\|A\|\widetilde{B})$$

is the 32-byte HMAC-SHA256 authentication tag, with A the (possibly empty) AAD (CVF2 fix: this tag definition now matches Table 1 exactly and explicitly binds A, whereas an earlier revision's inline formula, $T = \mathrm{Derive}_{0x03}(\mathtt{kb}, N\|\widetilde{B})$, omitted the AAD and was — read through the then-stated nonce-first Derive_d formula — inconsistent with the AAD-binding tag the reference code actually computed). The minimum valid ciphertext length is 48 bytes.

3.8 Online-AE Streaming Format (v6s-ae)

The basic streaming format (`encrypt_stream`) carries a single tag at the end of the stream, causing `decrypt_stream` to release plaintext before the tag is verified (RUP, formerly CAV-001). The online-AE streaming format eliminates this by attaching a per-chunk HMAC-SHA256 tag to each segment of the masked varint blob.

$$S = N \parallel \Big[\mathrm{be4}(\ell_i)\|\widetilde{C}_i\|T_i\Big]_{i=0}^{n-1} \parallel \mathrm{be4}(0)\|T_{\mathrm{final}},$$

where each $\widetilde{C}_i$ is at most 1024 bytes of the masked varint blob and (CVF2 fix: nonce moved to the fixed byte 1..16 offset shared with every other domain):

$$T_i = \mathrm{HMAC}\Big(\mathtt{kb},\ \mathtt{0x08}\|N\|\mathrm{be4}(|A|)\|A\|\mathrm{be4}(i)\|\widetilde{C}_i\Big)\,,$$
$$T_{\mathrm{final}} = \mathrm{HMAC}(\mathtt{kb},\ \mathtt{0x09}\|N\|\mathrm{be4}(|A|)\|A\|\mathrm{be4}(n))\,.$$

The sentinel frame has $\ell = 0$; it signals end-of-stream. `decrypt_stream_ae` verifies T_i before yielding any plaintext from chunk i. T_{final} binds the total chunk count n, preventing silent truncation at a chunk boundary. Chunk index i in each T_i prevents chunk-reordering attacks.

3.9 Format Applicability and Normative Status (CVF7 fix)

Remark 3.15 (Audit finding CVF7)**.** *The preceding subsections define three distinct wire encodings — block-mode Wire Format Version 7 ($C = N\|\widetilde{B}\|T$, Section 3.7), the basic streaming format (`encrypt_stream`/`decrypt_stream`, a single trailing tag), and the online-AE streaming format (v6s-ae, Section 3.8, per-chunk tags) — without ever stating their relationship: whether they are interchangeable options, whether block-mode and streaming coexist by design, or how a decryptor determines which of the three a given ciphertext uses, since none of the three wire layouts carries a version/format discriminator byte. This was audit finding **CVF7**. Because the basic streaming format is RUP-prone (CAV-001, Section 11), leaving the relationship unstated*

has a security consequence: absent a stated selection rule, a receiver could be induced to invoke the unsafe decoder on a ciphertext produced by (or claimed to come from) the safe encoder. The remainder of this subsection states the rule that was missing.

Block mode and streaming mode coexist by design; both are normative. They are not alternative encodings of one message class chosen by preference — they serve two disjoint use cases:

- **Block mode** (Wire Format v7, Section 3.7) is normative for bounded messages that fit in memory and require length-bucket obfuscation via padding. It is the sole format underlying Enc/Dec (Definition 3.17) and the sole format covered by the IND-CPA (Theorem 4.7) and INT-CTXT theorems of Section 4.
- **Streaming mode** is normative for unbounded or memory-constrained plaintext that cannot be buffered whole. It never applies block padding, so a streaming ciphertext always discloses the exact plaintext length and is never interoperable with a block-mode ciphertext (different framing, different token encoding; Section 3.8).

Within streaming mode, only the online-AE format is normative; the basic streaming format is deprecated and forbidden for new ciphertexts.

- **The online-AE streaming format (v6s-ae, Section 3.8) is, as of this fix, the sole normative streaming format.** It MUST be used for every new streaming deployment.
- **The basic streaming format (`encrypt_stream/decrypt_stream`) is superseded, deprecated, and its use is forbidden for producing new ciphertext.** It is retained solely so that streams produced before this fix remain decryptable; no protocol may be designed to accept it for new traffic, and no new ciphertext may be issued in this format. This upgrades CAV-001 (Section 11) from "fixed, with the RUP-prone format still permitted at the caller's discretion" to "fixed by supersession: the RUP-prone format is deprecated and forbidden going forward."

Format selection is an out-of-band API contract, not an in-band discriminator. None of the three wire layouts carries a version/format byte, and this fix does not add one. Exactly as with key and nonce agreement, the two endpoints of a NAPQES deployment agree out-of-band, at the protocol/call-site level, on which of block mode, `encrypt_stream_ae/decrypt_stream_ae`, or the deprecated `encrypt_stream/decrypt_stream` is in use for a given channel or message class — the same way a caller of any AEAD library chooses between a one-shot and a streaming construction rather than having the library sniff the format from ciphertext bytes. This is a deliberate design choice and not an omission: a self-describing in-band discriminator would itself have to be authenticated before a decryptor could safely act on it, which reintroduces the same trust-before-verification problem RUP already poses. Three consequences follow: (1) a decryptor is never expected to auto-detect format from ciphertext content alone; (2) a protocol built on NAPQES MUST NOT implement a "try v6s-ae, then fall back to the basic streaming format" negotiation, since that downgrade path would recreate exactly the induced-unsafe-decode hazard this fix closes; and (3) any future in-band discriminator, should one be added, would itself require authentication under the tag before a decryptor may act on it — that mechanism is deferred as out of scope for this fix.

3.10 The NAPQES AEAD Algorithm Triple

Remark 3.16 (Audit finding CVF5)**.** *The subsections above state notation, key generation, per-domain HMAC derivation, noise probability, padding, the token-emission loop, and the wire format,*

but nowhere assembled them into the formal algorithm triple that an AEAD definition requires (cf. the Ascon v1.2 submission [3]), and Section 4 already invokes NAPQES.Enc *and* NAPQES.Dec *(e.g. the INT-CTXT proof, Section 4) without those symbols ever having been formally defined — a key-generation/setup algorithm, an encryption algorithm* $\mathsf{Enc}(K, N, A, M) \to C$*, and a decryption algorithm* $\mathsf{Dec}(K, N, A, C) \to M$ *or* $\perp$*, each with explicitly typed inputs/outputs and a stated correctness condition, were left implicit and scattered across the prose and pseudocode above. This was audit finding* ***CVF5****. Definition 3.17 below assembles the pieces already introduced into that triple; no algorithm, encoding, or derivation changes — this is a specification-assembly fix, not a code or wire-format change.*

Definition 3.17 (NAPQES AEAD algorithm triple)**.** *NAPQES is the triple* $(\mathsf{KeyGen}, \mathsf{Enc}, \mathsf{Dec})$ *over the following typed spaces:*

- *Key space* $\mathcal{K}$*: ordered* K*-tuples of distinct elements of* $\mathcal{P}$ *(Section 3, "Key").*
- *Nonce space* $\mathcal{N} = \{0,1\}^{128}$*.*
- *AAD space* $\mathcal{A} = \{0,1\}^*$*.*
- *Message space* $\mathcal{M}$*: finite sequences of Unicode codepoints (Python* `str` *values, read via* $\mathrm{ord}(\cdot)$*) of length at most* `MAX_PLAINTEXT_CODEPOINTS` $= 2^{16} - 1$*, the limit imposed by the 2-codepoint big-endian length prefix written during padding (Section 3, "Plaintext Padding").*
- *Ciphertext space* $\mathcal{C} = \{0,1\}^*$*; every output of* Enc *has length* $16 + 8N + 32$ *bytes, where* N *is the real-plus-noise token count for that message (minimum 48 bytes).*

$\mathsf{KeyGen}(1^\lambda) \to \mathbf{k}$**.** *Draw* K *distinct primes* $k_1, \ldots, k_K$ *uniformly without replacement from* $\mathcal{P}$ *by CSPRNG-driven rejection sampling (*`generate_prime_numbers`*); output* $\mathbf{k} = (k_1, \ldots, k_K) \in \mathcal{K}$ *and set* $\mathsf{kb} = \mathsf{key\_bytes}(\mathbf{k})$ *as in Section 3, "Key".*

$\mathsf{Enc}(\mathbf{k}, N, A, M) \to C$**.** *On input* $\mathbf{k} \in \mathcal{K}$*,* $N \in \mathcal{N}$*,* $A \in \mathcal{A}$*,* $M \in \mathcal{M}$*: (1) compute* kb*; (2) pad* M *to* P_d *("Plaintext Padding"); (3) run the token-emission loop over* P_d *using* (kb, N) *("Token Construction") to obtain the token sequence* $\mathrm{token}[0..N{-}1]$*; (4) serialise* $B = \mathrm{be8}(\mathrm{token}[0]) \| \cdots \| \mathrm{be8}(\mathrm{token}[N{-}1])$ *("Wire Format, Version 7"); (5) mask* $\widetilde{B} = B \oplus \mathrm{Derive}_{0x07}(\mathsf{kb}, N, \cdot)$*; (6) compute the tag* $T = \mathrm{Derive}_{0x03}(\mathsf{kb}, N, \mathrm{be4}(|A|) \| A \| \widetilde{B})$*; (7) output* $C = N \| \widetilde{B} \| T$*.*

This is the algorithm the audit's recommended signature, $\mathsf{Enc}(K, N, A, M) \to C$*, refers to, and it is exactly the internal, explicit-nonce encryption routine used by the FIPS power-on self-test and the KAT harness. The public, randomized API,* $\mathsf{Encrypt}(\mathbf{k}, A, M) \triangleq \mathsf{Enc}(\mathbf{k}, N, A, M)$ *for* $N \xleftarrow{\$} \{0,1\}^{128}$ *sampled internally by a CSPRNG (*`encrypt_bytes`*/*`encrypt_str`*), is a randomized wrapper around* Enc *that never exposes* N *as a caller-chosen input (the explicit-nonce* Enc *itself is not part of the public API — see* ***CVF3*** *for why caller-chosen nonces were restricted). This distinction — a deterministic, nonce-keyed core algorithm versus a randomized public wrapper that samples the nonce — is why nonce reuse (CVF3) is a DRBG-failure/replay hazard rather than a caller-misuse hazard in normal use, and why Theorem 4.7's experiment (which queries the public, randomized encryption oracle) and Lemma 4.15's per-nonce framing both apply to the same underlying* Enc*.*

$\mathsf{Dec}(\mathbf{k}, A, C) \to M$ **or** $\perp$**.** *On input* $\mathbf{k} \in \mathcal{K}$*,* $A \in \mathcal{A}$*,* $C \in \{0,1\}^*$*: (1) if* $|C| < 48$*, output* $\perp$*; (2) parse* $C = N \| \widetilde{B} \| T'$ *with* $N \in \{0,1\}^{128}$ *the leading 16 bytes and* T' *the trailing 32 bytes; (3) compute* kb *and* $T = \mathrm{Derive}_{0x03}(\mathsf{kb}, N, \mathrm{be4}(|A|) \| A \| \widetilde{B})$*; (4) if* $T \neq T'$ *(compared in constant time), output* $\perp$*; (5) else unmask* $B = \widetilde{B} \oplus \mathrm{Derive}_{0x07}(\mathsf{kb}, N, \cdot)$*, deserialise the fixed-width tokens, invert the token-emission loop using the same* (kb, N)*-derived noise-position oracle and addends to recover* P_d*, strip the length prefix and padding, and output* M*.*

We write Dec *as taking* $(\mathbf{k}, A, C)$ *rather than the audit's suggested* $(\mathbf{k}, N, A, C)$ *because NAPQES's wire format embeds* N *as* C*'s first 16 bytes (step 2 above) rather than transmitting it out-of-band; this matches the actual* `decrypt_bytes(ciphertext, key, aad)` *call signature and the* $\mathsf{Dec}(\mathbf{k}, c^*, aad^*)$ *notation already used in the INT-CTXT proof (Section 4), and is a trivial reparameterisation of the audit's four-argument form under* $N := C[0{:}16]$ *— no security-relevant difference results from embedding* N *in* C *versus passing it separately, since both are fully determined by the ciphertext, and the domain-separation argument of Remark 4.1 does not depend on which of the two calling conventions is used.*

Definition 3.18 (Correctness)**.** *For every* λ*, every* $\mathbf{k} \leftarrow \mathsf{KeyGen}(1^\lambda)$*, every* $N \in \mathcal{N}$*,* $A \in \mathcal{A}$*,* $M \in \mathcal{M}$*:*

$$\mathsf{Dec}(\mathbf{k},\ A,\ \mathsf{Enc}(\mathbf{k}, N, A, M)) = M.$$

This holds by construction: Enc*'s token-emission loop (step 3) and* Dec*'s loop-inversion step (step 5) both compute the noise-position oracle (domain* `0x00`*), the real/noise addends (domains* `0x01/0x05`*), and the padding codepoints (domain* `0x06`*) as pure, deterministic functions of* $(\mathsf{kb}, N, \text{position})$*; given the same* $\mathbf{k}$ *and* N*, both algorithms therefore compute identical intermediate values at every position, so token emission and token inversion are exact inverses whenever* $T = T'$*. Correctness is additionally checked empirically by the cross-language KAT corpus (*`tests/kat/v6_vectors.json`*) and the full regression suite across the Python, Rust, and C implementations (Section 7).*

Remark 3.19 (Authentication-failure output $\perp$)**.** *The formal* $\perp$ *output of* Dec *is realised in all three reference implementations as a raised exception (*`ValueError` *in Python/Rust/C-equivalent error codes) rather than a sentinel return value; SPEC.md §10 ("Error model") enumerates every condition under which* Dec *raises rather than returns* M*, including tag mismatch, undersized ciphertext, and truncated streaming input. This is a standard, semantically equivalent realisation of* $\perp$ *(cf. most software AEAD APIs, e.g.* `cryptography.hazmat`*), and every occurrence of "*$\mathsf{Dec}(\cdot) = \perp$*" or "*$\mathsf{Dec}(\cdot) \neq \perp$*" in Section 4 below should be read as, respectively, "raises" or "does not raise".*

3.11 V8 Key Schedule and Synthetic Nonce (CVF3 / CVF8 / CVF13 fix)

Remark 3.20 (Why v7 cannot close CVF3/CVF8/CVF13 without a key-schedule change)**.** *Three audit findings trace back to a single design choice shared by every wire format up to and including v7:* one *secret,* $\mathbf{k}$ *(via* $\mathsf{kb} = \mathsf{key\_bytes}(\mathbf{k})$*), plays two unrelated roles — it keys every domain-derivation HMAC call,* and *it is the arithmetic-layer prime tuple used directly by the token map* $c \mapsto c \cdot k + a$*.*

- ***CVF3.*** *Because every derived value is a deterministic function of* (kb, N)*, a repeated nonce reproduces an identical keystream and addends; combined with the exact affine token map, this permits exact key recovery from two known plaintexts under one reused nonce (footnote to Table 2). A CSPRNG-sampled nonce only makes* accidental *reuse improbable; it does nothing against DRBG failure, restart/replay, or process-snapshot reuse.*
- ***CVF8.*** kb *has only* $H_\infty(\mathbf{k}) \approx 19.16K$ *bits of min-entropy (Remark 4.2), a non-standard HMAC-key hypothesis distinct from the textbook uniform-key PRF conjecture.*
- ***CVF13.*** *A reduction simulating* Enc *must know* $\mathbf{k}$ *to run the arithmetic layer, yet the same* $\mathbf{k}$ *(via* kb*) is supposed to be the PRF oracle's hidden key — a reduction that already knows the "hidden" key cannot meaningfully distinguish it from random, so the PRF hop is vacuous as written (Remark 4.11).*

Remark 4.20 shows that deriving $\mathsf{sk} = \mathrm{HKDF}(\mathsf{kb})$ *— the fix originally sketched under CVF8 —* does not *resolve CVF13, because* sk *remains a deterministic function of* $\mathbf{k}$*: knowing* $\mathbf{k}$ *still means knowing* sk*. All three findings are closed simultaneously only by making the HMAC-keying secret*

and the arithmetic-layer secret independently sampled *random variables, with no deterministic function relating one to the other. This subsection specifies that construction (v8) precisely.*

Definition 3.21 (V8 key generation)**.** $\mathsf{KeyGen}_{v8}(1^\lambda) \to (\mathbf{k}, \mathsf{sk})$*: draw* $\mathbf{k} = (k_1, \ldots, k_K)$ *exactly as in* KeyGen *(Definition 3.17), and,* independently*, draw* $\mathsf{sk} \xleftarrow{\$} \{0,1\}^{256}$ *uniformly via CSPRNG (`rand::thread_rng().fill_bytes`, `generate_v8_key` in `rust/src/lib.rs`). No function relates* sk *to* $\mathbf{k}$ *or to* $\mathsf{kb} = \mathsf{key\_bytes}(\mathbf{k})$*; both* $\mathbf{k}$ *and* sk *are secret key material that MUST be generated, stored, and transmitted together.*

Definition 3.22 (Synthetic nonce — domain `0x0A`)**.** *For* $\mathsf{sk} \in \{0,1\}^{256}$*,* $A \in \{0,1\}^*$*,* $M \in \mathcal{M}$ *(encoded as UTF-8 bytes),*

$$N_{v8}(\mathsf{sk}, A, M) \;=\; \mathrm{HMAC}(\mathsf{sk},\; \texttt{0x0A} \,\|\, \mathrm{be4}(|A|) \,\|\, A \,\|\, M)[0{:}16].$$

Unlike domains `0x00`–`0x09`, this derivation takes no nonce as input — it is the step that produces *the nonce, in the manner of RFC 5297 SIV / AES-GCM-SIV's synthetic IV, and is keyed only by* sk*, never by* $\mathbf{k}$*.*

Definition 3.23 (V8 encryption and decryption)**.** $\mathsf{Enc}_{v8}(\mathbf{k}, \mathsf{sk}, A, M) \to C$*: (1) compute* $N = N_{v8}(\mathsf{sk}, A, M)$ *(Definition 3.22); (2) run* $\mathsf{Enc}(\mathbf{k}, N, A, M)$ *(Definition 3.17) verbatim, with every occurrence of* kb *in that algorithm's steps (2)–(6) replaced by* sk *— i.e. padding (domain `0x06`), the noise-position oracle (`0x00`), addends (`0x01`/`0x05`), noise characters (`0x04`), the `0x07` keystream, and the `0x03` tag are all keyed by* sk*; only the token map* $c \mapsto c \cdot k + a$ *itself still uses* $\mathbf{k}$ *directly, exactly as in* Enc*. Output* C*.* $\mathsf{Dec}_{v8}(\mathbf{k}, \mathsf{sk}, A, C) \to M$ *or* $\perp$*: run* $\mathsf{Dec}(\mathbf{k}, A, C)$ *verbatim with* kb *replaced by* sk *throughout. Correctness (Definition 3.18) carries over unchanged, since the substitution is uniform and the same* sk *is used by both algorithms. Implemented as `encrypt_bytes_v8`/ `decrypt_bytes_v8` in `rust/src/lib.rs`; the wire-format byte shape* $(N \| \widetilde{B} \| T)$ *is unchanged, but v7 and v8 ciphertexts are* not *interoperable with each other (different keying and nonce derivation), and — per the format-selection philosophy of Section 3.9 — callers must agree out-of-band on which schedule a given key/ciphertext pair uses.*

Lemma 3.24 (V8 closes CVF3: nonce reuse across distinct messages is negligible)**.** *For any two queries* $(A_1, M_1) \neq (A_2, M_2)$ *to* Enc_{v8} *under the same* sk*,* $\Pr[N_{v8}(\mathsf{sk}, A_1, M_1) = N_{v8}(\mathsf{sk}, A_2, M_2)]$ *is bounded by* $\mathrm{Adv}^{\mathrm{PRF}}_{\text{HMAC-SHA256}}(\mathcal{B}) + 2^{-128}$ *(a PRF hop identical in form to Lemma 4.12's hop, plus the birthday bound on the truncated 128-bit output), for a PRF adversary* $\mathcal{B}$ *with access to* sk *only via domain `0x0A` queries. Consequently the affine-cancellation key-recovery route of the Table 2 footnote — which requires two* distinct *known plaintexts under one* reused *nonce — applies to* Enc_{v8} *only with this same negligible probability, rather than with the* $\approx 2^{-64}$ *birthday probability of v7's independent CSPRNG nonce. Re-querying* Enc_{v8} *on an* identical (A, M) *pair reproduces the identical* N *(and hence the identical* C*) deterministically by Definition 3.22 — this discloses only plaintext equality, the standard, disclosed MRAE trade-off (cf. AES-GCM-SIV), not a key-recovery or confidentiality break.*

Lemma 3.25 (V8 closes CVF8: the standard uniform-key PRF assumption applies)**.** *By Definition 3.21,* $\mathsf{sk} \xleftarrow{\$} \{0,1\}^{256}$ *exactly, so* $H_\infty(\mathsf{sk}) = 256$ *and* $\mathrm{Adv}^{\mathrm{PRF}}_{\text{HMAC-SHA256}}(\mathcal{B})$ *against* sk *is the conventional, uniform-key HMAC-SHA256 PRF advantage the literature states — no key-guessing term* $q_F \cdot 2^{-H_\infty(\cdot)}$ *(Remark 4.2) and no PRF-under-non-uniform-D caveat (Remark 4.10) is needed for a v8-restated Theorem 4.7: simply replace* kb *by* sk *throughout the proof of Theorem 4.7 and drop the* $q_F \cdot 2^{-H_\infty(\mathbf{k})}$ *term, since* $\mathbf{k}$ *no longer keys any HMAC call the adversary can query.*

Lemma 3.26 (V8 closes CVF13: the reductions are genuinely simulatable)**.** *Let* $\mathcal{O}$ *be a PRF challenger's oracle, keyed either by a uniform* $\mathsf{sk}^* \xleftarrow{\$} \{0,1\}^{256}$ *(real world) or by a truly random function (ideal world). Construct* $\mathcal{B}$ *as follows:* $\mathcal{B}$ *samples its own* $\mathbf{k}'$ *via* KeyGen_{v8}*'s prime-sampling step (Definition 3.21), independently of* $\mathcal{O}$*; runs* $\mathsf{Enc}_{v8}(\mathbf{k}', \mathsf{sk}^*, A, M)$ *for each of* $\mathcal{A}$*'s queries by computing the arithmetic layer* $(c \cdot k'_j + a)$ *itself using* $\mathbf{k}'$*, and forwarding every domain-`0x00`–`0x0A` derivation call to* $\mathcal{O}$ *in place of directly computing* $\mathrm{HMAC}(\mathsf{sk}^*, \cdot)$*. Because* sk^* *is sampled independently of* $\mathbf{k}'$ *(Definition 3.21),* $\mathcal{B}$ *never needs to know* sk^* *to run this*

simulation — unlike the v7 case (Remark 4.11) and unlike the rejected $\mathsf{sk} = \mathrm{HKDF}(\mathsf{kb})$ *repair (Remark 4.20), where knowing the arithmetic key implied knowing the HMAC key. When* $\mathcal{O}$ *is real,* $\mathcal{B}$*'s simulation is distributed identically to* $\mathsf{Enc}_{v8}(\mathbf{k}, \mathsf{sk}^*, \cdot, \cdot)$ *for the true, independently-sampled* $\mathbf{k}$ *(both are: a uniformly-drawn prime tuple, together with* Enc_{v8} *run under the real* sk^* *— Lemma 4.12/Lemma 4.15 already establish that the adversary's view depends on* $\mathbf{k}$ *only through the* sk^**-masked blob and the content-independent token* count*, not through* $\mathbf{k}$*'s specific value); when* $\mathcal{O}$ *is ideal, it is distributed identically to the* G_1 *hop of Theorem 4.7/Theorem 4.24. Hence* $|\Pr[\mathcal{A} \text{ wins real}] - \Pr[\mathcal{A} \text{ wins ideal}]| \le \mathrm{Adv}^{\mathrm{PRF}}_{\text{HMAC-SHA256}}(\mathcal{B})$*, exactly the PRF hop Theorem 4.7 and Theorem 4.24 require — the simulation gap Remark 4.11 identified for v7 does not arise for v8.*

Remark 3.27 (Scope and residual)**.** *Lemmas 3.24–3.26 close CVF3, CVF8, and CVF13 for callers who adopt the v8 key schedule. The v7 wire format, key schedule, and their previously-documented residuals (Remarks 4.19, 4.20, 4.11) are unchanged and remain available for backward compatibility; v7 callers who need the stronger guarantees above must migrate to* KeyGen_{v8}/Enc_{v8}/Dec_{v8}*. The v8 schedule is implemented in* `rust/src/lib.rs` *(*`generate_v8_key`*,* `encrypt_bytes_v8`*,* `decrypt_bytes_v8`*) with unit tests covering round-trip correctness, tamper/AAD-mismatch rejection, same-message determinism (the disclosed MRAE trade-off), and distinct-message nonce distinctness. The Python (*`napqes.py`*) and C (*`C/napqes.c`*) reference implementations have not yet been ported to v8; this is tracked in* `docs/CAVEATS.md` *as a follow-up so all three languages offer the misuse-resistant path.*

4 Security Analysis

Theorem 4.7 (IND-CPA), the IND-CCA discussion of Section 4.6, and the INT-CTXT theorem below are all stated against the formal $(\mathsf{KeyGen}, \mathsf{Enc}, \mathsf{Dec})$ triple of Definition 3.17 and assume the correctness condition of Definition 3.18; together, INT-CTXT (ciphertext integrity against an adversary with encryption-oracle access) and the underlying HMAC-SHA256 tag's collision/forgery resistance are the EUF-CMA-style unforgeability goal referenced informally in Section 1 — NAPQES does not define a standalone MAC verification key separate from kb, so EUF-CMA unforgeability of the tag and INT-CTXT unforgeability of the ciphertext coincide here. Arguments below that elide N or A (e.g. writing $\mathsf{Enc}(\mathbf{k}, m)$) do so only where the elided argument is fixed or immaterial to the hybrid being analysed; the fully-typed calls are always as given in Definition 3.17.

4.1 IND-CPA Security

Remark 4.1 (Domain Separation of HMAC Inputs)**.** *As of the CVF2 fix, every internal NAPQES derivation invokes* $F(\mathsf{kb}, \cdot)$ *with an input of the single, uniform shape*

$$d \,\|\, N \,\|\, \mathsf{ctx},$$

where $d \in \{$`0x00`$, \ldots,$ `0x09`$\}$ *is a 1-byte domain separator,* N *is the 16-byte nonce (always at byte offset* 1..16*), and* ctx *is either a fixed-width positional counter (domains* $0, 1, 4, 5$*: 5 B; domains* $6, 7$*: 4 B; domain* 2*: empty) or a length-prefixed, AAD-binding suffix (domains* $3, 8, 9$*:* $\mathrm{be4}(|A|)\|A\|\cdot$*).*

Cross-domain distinctness (unconditional). *Two inputs* $d_1\|N_1\|\mathsf{ctx}_1$ *and* $d_2\|N_2\|\mathsf{ctx}_2$ *with* $d_1 \neq d_2$ *differ at byte position* 0 *and are therefore always distinct, regardless of* N *or* ctx *— no probabilistic argument is required.*

Intra-domain injectivity. *Fix a domain* d*. Bytes* 1..16 *of every input to* Derive_d *are the nonce* N*, so for a fixed* N *the map* $\mathsf{ctx} \mapsto d\|N\|\mathsf{ctx}$ *is injective as long as* $\mathsf{ctx} \mapsto \mathsf{ctx}$ *itself is injective. For domains* $0, 1, 4, 5, 6, 7$*,* ctx *is a fixed-width big-endian counter, so this holds trivially. For*

domains $3, 8, 9$, $\mathsf{ctx} = \mathrm{be4}(|A|)\|A\|\mathsf{rest}$*: because* $\mathrm{be4}(|A|)$ *fixes* $|A|$*, the split between* A *and* rest *is unambiguous, so distinct* (A, rest) *pairs never collide — the length prefix makes the AAD-binding encoding injective. Consequently, for a fixed nonce* N*, distinct semantically-intended derivations under the same domain always query* $F(\mathsf{kb}, \cdot)$ *at distinct inputs, and (by the cross-domain argument) derivations under different domains never collide either. No cross-group probabilistic term (as required under the pre-CVF2 mixed nonce-first/domain-first layout) is needed.*

In the remainder of Section 4 we treat all $F(\mathsf{kb}, \cdot)$ *evaluations as queries to a single PRF; the domain-separation argument above guarantees that semantically distinct derivations always query the PRF at distinct inputs.*

Remark 4.2 (Audit finding CVF8: the bound must depend on key entropy)**.** *The original statement of Theorem 4.7 bounded* $\mathrm{Adv}^{\text{IND-CPA}}_{\text{NAPQES}}(\mathcal{A})$ *purely in terms of* $\mathrm{Adv}^{\text{PRF}}_{\text{HMAC-SHA256}}(\mathcal{B}_1)$ *and a nonce-collision term, with no term depending on* K *or* $|\mathcal{P}|$*. This is unsound:* kb *is not a uniformly random bit-string, it is* $\mathsf{key\_bytes}(\mathbf{k})$ *for* $\mathbf{k}$ *drawn from the* $2^{H_\infty(\mathbf{k})}$*-point prime-tuple distribution of the Key subsection, and the standard HMAC-SHA256 PRF conjecture is stated for a uniformly random key. An adversary can always recover* $\mathbf{k}$ *(and thereafter win the IND-CPA game outright) by exhaustively evaluating* $\mathrm{HMAC}(\cdot, \cdot)$ *offline against a list of candidate keys, at cost* $\approx 2^{H_\infty(\mathbf{k})}$ *evaluations in the worst case; this attack is entirely outside the uniform-key PRF game and so is invisible to a bound stated only in terms of* $\mathrm{Adv}^{\text{PRF}}_{\text{HMAC-SHA256}}$*. For small* K *(e.g.* $K = 1$*,* $H_\infty(\mathbf{k}) \approx 19$–$20$ *bits) this search is trivial, so the original theorem was false as stated for admissible small key sizes, and imprecise even at the paper's default* $K = 10$*. The restated theorem below adds the missing key-guessing term* $q_F \cdot 2^{-H_\infty(\mathbf{k})}$ *(Lemma 4.8) and Remark 4.17 gives the minimum* K *the theorem requires to meet a target security level.*

Remark 4.3 (Audit finding CVF9: INDCPA was never formally defined, and the challenge constraint was ill-specified)**.** *The original text stated Theorem 4.7 and its proof against "the standard IND-CPA experiment" without ever giving a formal IND-CPA definition (adversary, oracles, challenge, advantage) — the experiment was only sketched inline inside the proof of Game* G_0 *below. That sketch then imposed a challenge constraint, "*$\mathcal{A}$ *submits a challenge pair* (m_0, m_1) *of* equal padded length*", which is non-standard (the textbook constraint is* $|m_0| = |m_1|$*, equal* plaintext *length, with no padding-related precondition) and did not state which of three genuinely different quantities — codepoint count, on-wire byte length, or the padded bucket* B *(Plaintext Padding subsection) — "length" referred to. Left unresolved, this made the theorem statement not well-defined, and let the hiding lemma's proof (Lemma 4.12) later invoke "equal padded lengths by the IND-CPA requirement" as an unproved assumption to close a step that would otherwise be circular. Definition 4.4 below supplies the missing formal definition, using the standard constraint* $|m_0| = |m_1|$ *with the length metric stated explicitly (codepoints, matching how* $\mathcal{M}$ *is typed in Definition 3.17 — the only metric NAPQES's algorithm triple is defined against); no "equal padded length" precondition is imposed on* $\mathcal{A}$*. Remark 4.5 then shows that equal padded length is a* consequence *of this standard constraint here (not an additional restriction smuggled in to make the proof close), because the padding bucket* B *is a deterministic function of codepoint count alone. Remark 4.6 states the residual scope limitation this leaves: the guarantee is with respect to* $\mathcal{M}$*'s codepoint metric, and does not, by itself, extend to external byte-string encodings of unequal codepoint count but equal byte length (audit finding CAV-003).*

Definition 4.4 (IND-CPA security)**.** *Let* $\Pi = (\mathsf{KeyGen}, \mathsf{Enc}, \mathsf{Dec})$ *be the NAPQES algorithm triple of Definition 3.17, and let* $\mathsf{Encrypt}(\mathbf{k}, A, M) \triangleq \mathsf{Enc}(\mathbf{k}, N, A, M)$ *for* $N \xleftarrow{\$} \mathcal{N}$ *be its randomized public encryption oracle (Section 3.10). For a PPT adversary* $\mathcal{A} = (\mathcal{A}_1, \mathcal{A}_2)$*, define the experiment* $\mathbf{Exp}^{\text{ind-cpa}}_{\Pi,\mathcal{A}}(\lambda)$*:*

1. $\mathbf{k} \leftarrow \mathsf{KeyGen}(1^\lambda)$.
2. $\mathcal{A}_1$ *is given adaptive oracle access to* $\mathsf{Encrypt}(\mathbf{k}, \cdot, \cdot)$ *and, after at most* q *total queries (counted jointly with step 4), outputs* $(A^*, m_0, m_1, \mathsf{st})$ *with* $m_0, m_1 \in \mathcal{M}$ *subject to exactly*

one constraint:

$$|m_0| = |m_1|,$$

where $|\cdot|$ denotes the number of Unicode codepoints of a message in $\mathcal{M}$ — the metric $\mathcal{M}$ is typed against in Definition 3.17. No other restriction (padded length, byte length, or otherwise) is imposed on $\mathcal{A}_1$'s choice of m_0, m_1.

3. *The challenger samples $b \xleftarrow{\$} \{0,1\}$ and returns $c^* \leftarrow \mathsf{Encrypt}(\mathbf{k}, A^*, m_b)$.*
4. *$\mathcal{A}_2(\mathsf{st}, c^*)$, with continued oracle access to $\mathsf{Encrypt}(\mathbf{k}, \cdot, \cdot)$, outputs a bit b'.*
5. *The experiment outputs 1 ("$\mathcal{A}$ wins") iff $b' = b$.*

The advantage of $\mathcal{A}$ is

$$\mathrm{Adv}^{\text{IND-CPA}}_{\text{NAPQES}}(\mathcal{A}) \;=\; \Big|\Pr\big[\mathbf{Exp}^{\text{ind-cpa}}_{\Pi,\mathcal{A}}(\lambda) = 1\big] - \tfrac{1}{2}\Big|,$$

and Π is IND-CPA secure if this quantity is negligible in λ for every PPT $\mathcal{A}$. This is the standard left-or-right IND-CPA notion for a symmetric encryption scheme, stated against NAPQES's typed algorithm triple with no bespoke preconditions; Theorem 4.7 is proved against exactly this definition, and q there is the total query count of this definition.

Remark 4.5 (Equal padded length is derived, not assumed)**.** *Definition 4.4 imposes only the standard constraint $|m_0| = |m_1|$ (equal codepoint count); it does not restrict $\mathcal{A}$ to equal* padded *length. The Plaintext Padding subsection shows $B = \max(16, 2^{\lceil \log_2(n+1) \rceil})$ is a deterministic function of $n = |P|$ (the codepoint count) alone, with no dependence on codepoint* values*. Consequently $|m_0| = |m_1|$ already forces m_0 and m_1 to share the same padding bucket B and the same padded length $B + 2$, with certainty — this is a logical consequence of Definition 4.4's standard constraint together with the padding formula, not an independent assumption. Every subsequent occurrence of "m_0, m_1 have equal padded length" below (Game G_0's challenge step, Lemma 4.12's proof) refers to this derived fact, closing the circularity audit finding CVF9 identified in the original text, which instead stated equal padded length as an unproved precondition and later invoked it as if it were the definition's own requirement.*

Remark 4.6 (Scope: the metric is codepoints, not on-wire bytes (residual of CVF9))**.** *Definition 4.4 measures $|m_0| = |m_1|$ in Unicode codepoints because that is the only metric $\mathcal{M}$ is formally typed against (Definition 3.17); this is not a free choice among equally natural alternatives, since NAPQES's* Enc *takes a codepoint sequence, not a byte string, as its message input. It follows that Theorem 4.7 says nothing about a caller-level scenario in which two externally byte-equal-length plaintexts (e.g. two UTF-8 byte strings of the same length) decode to* different *codepoint counts: such a pair falls outside $\mathcal{A}$'s admissible challenge set entirely (they are not even a valid $(m_0, m_1) \in \mathcal{M} \times \mathcal{M}$ challenge unless re-expressed at the codepoint level), and if instead compared as codepoint-level messages of unequal length, the padded bucket B — and hence ciphertext length — may differ between them, consistent with the length-bucket leak already documented as CAV-003. A caller relying on NAPQES to hide the distinction between two plaintexts that are equal-length only in bytes, not in codepoints, is not covered by Theorem 4.7 and should consult CAV-003 (Section 11).*

Theorem 4.7 (IND-CPA)**.** *Under the PRF assumption on HMAC-SHA256 (Remark 4.10 makes precise which key distribution this assumption is invoked against once the guessing term below has been paid for, and Remark 4.11 makes precise an additional simulation-gap caveat the reduction $\mathcal{B}_1$ is subject to), the NAPQES block encryption scheme is IND-CPA secure (Definition 4.4) provided K and $|\mathcal{P}|$ are large enough that $H_\infty(\mathbf{k})$ meets the target security level (Remark 4.17). Specifically, for any PPT adversary $\mathcal{A}$ making at most q encryption queries and at most q_F offline evaluations of* $\mathrm{HMAC}(\cdot,\cdot)$ *under adversarially-chosen keys, there exists a PRF adversary $\mathcal{B}_1$ with similar running time such that:*

$$\mathrm{Adv}^{\text{IND-CPA}}_{\text{NAPQES}}(\mathcal{A}) \le \mathrm{Adv}^{\text{PRF}}_{\text{HMAC-SHA256}}(\mathcal{B}_1) + \frac{q^2}{2^{128}} + q_F \cdot 2^{-H_\infty(\mathbf{k})}.$$

Proof. We prove the bound via a sequence of two hybrid games.

Game G_0 (Real Game). This is the standard IND-CPA experiment. The challenger samples $\mathbf{k} = (k_1, \ldots, k_K)$ uniformly from K-element ordered tuples of distinct primes in $\mathcal{P}$, computes $\mathsf{kb} = \mathsf{key\_bytes}(\mathbf{k})$, and answers up to q chosen-plaintext queries honestly using the real NAPQES.Enc algorithm (which calls $F(\mathsf{kb}, \cdot) = \text{HMAC-SHA256}(\mathsf{kb}, \cdot)$ for all internal derivations). After the query phase, $\mathcal{A}$ submits a challenge pair (m_0, m_1) with $|m_0| = |m_1|$ (Definition 4.4) — which, by Remark 4.5, already forces m_0, m_1 to have equal padded length; no separate precondition is imposed here. The challenger samples $b \xleftarrow{\$} \{0,1\}$, returns $c^* \leftarrow \text{NAPQES.Enc}(\mathbf{k}, m_b)$, and $\mathcal{A}$ wins if its output $b' = b$.

Game G_1 (PRF Replacement). Identical to G_0, except the function $F(\mathsf{kb}, \cdot)$ is replaced by a truly random function $\rho : \{0,1\}^* \to \{0,1\}^{256}$, sampled uniformly at the start of the experiment and evaluated lazily. Every call that would invoke $\text{HMAC}(\mathsf{kb}, x)$ now returns $\rho(x)$ instead.

Lemma 4.8 (Key-guessing bound (audit finding CVF8))**.** *Let q_F upper-bound the number of evaluations of* $\text{HMAC}(\kappa, \cdot)$ *that $\mathcal{A}$ performs* offline*, under keys κ of its own choosing, in addition to its q online encryption queries (this is exactly the resource the original theorem statement left unbounded — see Remark 4.2). Let* Guess *denote the event that some such κ equals the challenger's actual* $\mathsf{kb} = \mathsf{key\_bytes}(\mathbf{k})$*. Since $\mathbf{k}$ is sampled uniformly from the $2^{H_\infty(\mathbf{k})}$ ordered K-tuples of distinct primes in $\mathcal{P}$ (Key subsection) independently of $\mathcal{A}$'s (unbounded-time, but q_F-query) offline guessing strategy, a union bound over the q_F guesses gives*

$$\Pr[\text{Guess}] \;\le\; q_F \cdot 2^{-H_\infty(\mathbf{k})}.$$

If Guess *occurs, $\mathcal{A}$ can compute $F(\mathsf{kb}, \cdot)$ directly and simulate the challenger perfectly, winning G_0 with probability* 1*; this is the exhaustive-key-search attack the original, entropy-free bound could not see. We therefore only bound $\mathcal{A}$'s advantage in G_0 conditioned on* $\overline{\text{Guess}}$ *via the PRF hop below, and add* $\Pr[\text{Guess}]$ *back in when combining the bounds.*

Lemma 4.9. *There exists a PRF adversary $\mathcal{B}_1$, running in time $\mathcal{O}(T_\mathcal{A})$ with at most $q \cdot L_{\max}$ oracle calls (where $L_{\max}$ is an upper bound on the number of HMAC evaluations per encryption), such that*

$$\left|\Pr[\mathcal{A} \text{ wins } G_0] - \Pr[\mathcal{A} \text{ wins } G_1]\right| \;\le\; \text{Adv}_F^{\text{PRF}}(\mathcal{B}_1).$$

Proof. $\mathcal{B}_1$ is given a function oracle $\mathcal{O}(\cdot)$ that is either $F(\mathsf{kb}, \cdot)$ (real) or $\rho(\cdot)$ (random). $\mathcal{B}_1$ simulates the IND-CPA experiment for $\mathcal{A}$, answering every internal HMAC call by forwarding to $\mathcal{O}$, and (see Remark 4.11 for why this description alone is incomplete, and how the remaining gap is resolved) samples $\mathbf{k}$ itself in order to run the arithmetic layer of NAPQES.Enc. If $\mathcal{O} = F(\mathsf{kb}, \cdot)$, the simulation is identical to G_0; if $\mathcal{O} = \rho$, it is identical to G_1. Hence $\mathcal{B}_1$'s distinguishing advantage equals $|\Pr[\mathcal{A} \text{ wins } G_0] - \Pr[\mathcal{A} \text{ wins } G_1]|$, conditioned throughout on $\overline{\text{Guess}}$ (Lemma 4.8). □

Remark 4.10 (What $\text{Adv}^{\text{PRF}}_{\text{HMAC-SHA256}}(\mathcal{B}_1)$ means here)**.** *Conditioned on* $\overline{\text{Guess}}$*, Lemma 4.9 shows that distinguishing G_0 from G_1 is exactly as hard as distinguishing $F(\mathsf{kb}, \cdot)$ from a random function* for kb drawn from the prime-tuple distribution D of the Key subsection, conditioned on not having been one of $\mathcal{A}$'s q_F offline guesses — *not for* kb *drawn uniformly from* $\{0,1\}^{40K}$*. These are different hypotheses: the HMAC-SHA256 PRF conjecture, as conventionally stated and studied, concerns a uniformly random key. D is a highly structured, measure-zero subset of $\{0,1\}^{40K}$ (5-byte-aligned fields, each in a prime-valued range, no repeats, order-independent as a set) with min-entropy $H_\infty(\mathbf{k}) = \log_2 |\mathcal{P}|!/(|\mathcal{P}| - K)! \ll 40K$. We write* $\text{Adv}^{\text{PRF}}_{\text{HMAC-SHA256}}(\mathcal{B}_1)$ *in Theorem 4.7 to denote the PRF advantage against keys drawn from D (conditioned on* $\overline{\text{Guess}}$*) specifically — a strictly weaker, and far less studied, assumption than the uniform-key one —*

and we do not claim the standard uniform-key HMAC-SHA256 PRF assumption directly implies this quantity is negligible. Section 4.3 discusses two ways to remove this residual non-standard assumption and restore a reduction to the standard, uniform-key conjecture.

Remark 4.11 (Audit finding CVF13: $\mathcal{B}_1/\mathcal{B}_2$ cannot simulate NAPQES encryption from oracle access alone)**.** *Lemma 4.9's proof (and, symmetrically, $\mathcal{B}_2$'s construction in the proof of Theorem 4.24, Case 1) states that the reduction "simulates the ... experiment for $\mathcal{A}$, answering every internal HMAC call by forwarding to $\mathcal{O}/f$." This is necessary but not sufficient: to answer $\mathcal{A}$'s encryption queries at all, the reduction must run* NAPQES.$\mathsf{Enc}(\mathbf{k}, \cdot)$*, and that algorithm uses* $\mathbf{k}$ *itself* outside *of any HMAC call — the token emission step computes $c \cdot k_j + a$ (Section 3) and the admissible addend range $[1, k_j - 1]$ (Definitions 3.9–3.10) both depend on the specific prime k_j, not merely on* HMAC$(\mathsf{kb}, \cdot)$*'s output. Forwarding HMAC calls to an oracle $\mathcal{O}$ does not supply* $\mathbf{k}$*: in the standard PRF game $\mathcal{O}$'s key is sampled uniformly and kept secret from the distinguisher, so $\mathcal{B}_1$ (resp. $\mathcal{B}_2$) has no way to extract* $\mathbf{k}$ *from oracle access, and cannot run the arithmetic layer as written.*

The reduction cannot be repaired by having $\mathcal{B}_1$ sample its own $\mathbf{k}'$ *locally for the arithmetic layer: whenever $\mathcal{O} = F(\mathsf{kb}, \cdot)$ (the real-world branch), there is no reason* kb *equals* $\mathsf{key\_bytes}(\mathbf{k}')$ *for the* specific $\mathbf{k}'$ *that $\mathcal{B}_1$ chose —* $\mathbf{k}'$ *and the PRF challenger's hidden key are independent random variables. The resulting simulation would answer $\mathcal{A}$'s queries with tokens computed under* $\mathbf{k}'$ *but tags/addends/positions derived from $F(\mathsf{kb}, \cdot)$ for an unrelated* $\mathbf{k}$*: an internally inconsistent hybrid that is a different scheme from* NAPQES.$\mathsf{Enc}(\mathbf{k}', \cdot)$ *for any single key, and so does not establish* $\Pr[\mathcal{A}$ *wins* $G_0]$ *as claimed. Symmetrically, if $\mathcal{B}_1$ instead supplies its own* $\mathbf{k}'$ to *the PRF challenger (a "chosen-key" PRF game, so that $\mathcal{O}$'s hidden key really is* $\mathsf{key\_bytes}(\mathbf{k}')$*), $\mathcal{B}_1$ already knows the tested key and can compute $F(\mathsf{key\_bytes}(\mathbf{k}'), x)$ itself, making its distinguishing advantage against $\mathcal{O}$ trivially ≈ 1 regardless of whether $\mathcal{O}$ is real or random — this shows nothing, since the premise of a PRF game (the key is hidden from the distinguisher) is violated by construction whenever the same key material must also be disclosed, in the clear, to run the arithmetic layer.*

This defect is shared verbatim by the INT-CTXT reduction $\mathcal{B}_2$ (Theorem 4.24's proof, Case 1), which likewise must run NAPQES.Enc *to answer $\mathcal{A}$'s q encryption queries before evaluating any forgery, and it therefore propagates into the IND-CCA bound (Theorem 4.30), which is assembled from $\mathcal{B}_1$'s and $\mathcal{B}_2$'s advantages via the Bellare–Namprempre composition [13]: neither advantage is actually established for the real NAPQES scheme as the reductions are currently written, so the composed IND-CCA bound rests on the same unproven premise. This is a distinct defect from CVF8/Remark 4.10 (which concerns the* key distribution *the PRF assumption is stated over) — CVF13 concerns whether the reduction can be run* at all*, independent of which key distribution is assumed.*

Resolution. *Remark 4.19's KDF-subkey design — deriving an independent, uniformly-distributed* $\mathsf{sk} = \mathrm{HKDF}(\mathsf{kb})$ *and keying every domain derivation with* sk *in place of* kb*, while* $\mathbf{k}$ *itself is used only for the (public, non-HMAC) arithmetic layer — resolves both defects simultaneously: once* sk *is architecturally independent of* $\mathbf{k}$*, $\mathcal{B}_1/\mathcal{B}_2$ may legitimately sample* $\mathbf{k}$ *locally (it is no longer correlated with the tested key at all, so no "chosen-key" game or coincidence is needed) while forwarding every domain-derivation call to an external oracle keyed by the PRF challenger's independent, hidden* sk*. Absent that change, the reductions as written require an explicit, additional* key-indistinguishability *step that this proof does not supply: a demonstration that a simulator holding* $\mathbf{k}$ *in the clear (for arithmetic) still cannot distinguish* HMAC$(\mathsf{key_bytes}(\mathbf{k}), \cdot)$ *from random using that knowledge alone — i.e., that* $\mathbf{k}$*'s use in the arithmetic layer leaks nothing that helps predict HMAC outputs under* $\mathsf{key\_bytes}(\mathbf{k})$*. We do not supply that step here: it is not implied by the standard HMAC-PRF conjecture (which is silent about a distinguisher that possesses the key), and we are not aware of a suitable weaker primitive-level assumption. Theorem 4.7, Theorem 4.24, and Theorem 4.30 should therefore be read as conditioned on this gap*

until either the KDF-subkey design lands or the missing key-indistinguishability step is supplied; we flag this as an open residual, cross-referenced from Section 4.3 and logged in Section 11.

Nonce collision probability. Let Coll denote the event that at least two of the $q+1$ nonces generated across the q CPA oracle calls and the challenge encryption are equal. Each nonce is drawn independently and uniformly from $\{0,1\}^{128}$, so by the union bound:

$$\Pr[\text{Coll}] \;\le\; \binom{q+1}{2} \cdot 2^{-128} \;\le\; \frac{q^2}{2^{128}}.$$

Lemma 4.12. *In G_1 conditioned on $\overline{\text{Coll}}$:*

$$\Pr[\mathcal{A} \textit{ wins } G_1 \mid \overline{\text{Coll}}] \;=\; \tfrac{1}{2}.$$

Proof. Fix any adversary view V arising from the CPA query phase, which uses nonces $N_1, \ldots, N_q$. Let N^* be the challenge nonce. Under $\overline{\text{Coll}}$, $N^* \notin \{N_1, \ldots, N_q\}$.

In G_1, every challenge-phase derivation uses an input prefixed by the domain byte d and then N^* (e.g., $\rho(\texttt{0x00}\|N^*\|\text{be5}(j))$, $\rho(\texttt{0x01}\|N^*\|\text{be5}(j))$, $\rho(\texttt{0x02}\|N^*)$, and so on for domains `0x03`–`0x07`). All CPA-phase evaluations used inputs containing some $N_i \neq N^*$ at the same fixed byte 1..16 offset. Since ρ is a truly random function, its outputs on distinct inputs are independent, uniform 256-bit values. Under $\overline{\text{Coll}}$, the challenge inputs are disjoint from all CPA-phase inputs, so all challenge ρ-outputs are fresh, uniform, and independent of V.

In particular, the keystream masking the token blob (domain `0x07`) is:

$$\text{ks}(N^*) = \rho(\texttt{0x07}\|N^*\|\text{be4}(0)) \,\|\, \rho(\texttt{0x07}\|N^*\|\text{be4}(1)) \,\|\, \cdots \quad \text{(truncated to } |B| \text{ bytes)},$$

where B denotes the fixed-width-encoded token sequence (Section 3.7).

Independence of $\text{ks}(N^*)$ from B (OTP argument). B is assembled entirely from ρ-outputs queried under domains $\{\texttt{0x00}, \texttt{0x01}, \texttt{0x02}, \texttt{0x04}, \texttt{0x05}, \texttt{0x06}\}$:

- Noise/real positions: $\rho(\texttt{0x00}\|N^*\|\text{be5}(\cdot))$
- Real/noise addends: $\rho(\texttt{0x01}\|N^*\|\text{be5}(\cdot))$, $\rho(\texttt{0x05}\|N^*\|\text{be5}(\cdot))$
- Noise probability: $\rho(\texttt{0x02}\|N^*)$
- Noise codepoints: $\rho(\texttt{0x04}\|N^*\|\text{be5}(\cdot))$
- Padding: $\rho(\texttt{0x06}\|N^*\|\text{be4}(\cdot))$

Each keystream block uses input $\texttt{0x07}\|N^*\|\text{be4}(j)$, domain byte `0x07`. By Remark 4.1 (cross-domain distinctness, unconditional — every input above shares the single uniform $d\|N\|\texttt{ctx}$ shape), no keystream input $\texttt{0x07}\|N^*\|\text{be4}(j)$ equals any B-construction input: they differ at byte position 0 (`0x07` vs. `0x00`–`0x06`). No probabilistic cross-group term is needed — the argument is exact, not an estimate over q queries (this closes audit finding CVF2's cross-reference, issue #848, which tracked the removal of that probabilistic term). Since ρ is a truly random function, outputs at *distinct* inputs are mutually independent; therefore $\text{ks}(N^*)$ is statistically independent of B.

Consequently, the one-time-pad identity applies: for any fixed deterministic B,

$$\tilde{B} = B \oplus \text{ks}(N^*) \;\overset{\text{d}}{=}\; \text{Uniform}(\{0,1\}^{8|B|}).$$

Equal byte-length of B across m_0, m_1 (CVF1 fix). Write $B = \text{be8}(\text{token}[0])\|\cdots\|\text{be8}(\text{token}[N-1])$, so $|B| = 8N$ (Section 3.7). The token count N (real + noise tokens) is determined entirely by:

(i) the padded codepoint count, which is equal for m_0 and m_1 because $|m_0| = |m_1|$ (Definition 4.4) forces equal padded length (Remark 4.5 — a derived fact, not an assumed requirement), and (ii) the noise-position oracle $\rho(N^* \| \texttt{0x00} \| \text{be5}(\cdot))$, which depends only on N^* and $\texttt{kb}$ (via ρ) — *not* on the codepoint values being encrypted. Since the challenger uses the same N^* and the same key regardless of which of m_0, m_1 is encrypted, the sequence of noise/real decisions at each *ct_pos* is identical in both cases, so N — and hence $|B| = 8N$ — is identical for m_0 and m_1. Therefore $\tilde{B}$ is uniformly distributed over strings of a length that is a deterministic function of the (equal) padded length alone, independently of the plaintext bit b and the view V.

Remark 4.13 (Erratum: the v6 argument was false)**.** *In the retired v6 construction, B was instead* $\text{varint}(\text{token}[0]) \| \cdots \| \text{varint}(\text{token}[N-1])$*, and* $\text{varint}(\cdot)$ *is* not *a fixed-width encoding: its byte-length grows with* $\text{token} = c \cdot k + a$*, hence with the plaintext codepoint c. Equal padded length only fixes N; it does not fix $\sum_i |\text{varint}(\text{token}[i])|$, which depends on codepoint* values*. The original v6 proof asserted "B is the same length for both plaintexts m_0, m_1 (they have equal padded lengths by the IND-CPA requirement)" without justification — this step is false, as demonstrated by audit finding CVF1 with an explicit distinguisher (m_0 = U+0001 repeated vs. m_1 = U+FFFF repeated, equal padded length, ciphertexts of different length with overwhelming probability, IND-CPA advantage ≈ 1). The fixed-width encoding of Section 3.7 removes the dependency of $|B|$ on codepoint values, which is what restores the step above.*

Freshness of the authentication tag. The tag is computed as $T = \rho(\texttt{0x03} \| N^* \| \text{be4}(|aad|) \| aad \| \tilde{B})$ (CVF2 fix: domain $\texttt{0x03}$ uses the same unified $d \| N \| \texttt{ctx}$ layout as every other domain; see Table 1). The tag input has first byte $\texttt{0x03}$ and, at bytes 1..16, N^*. All CPA-phase tag inputs are of the form $\texttt{0x03} \| N_i \| \text{be4}(|aad_i|) \| aad_i \| \tilde{B}_i$ with $N_i \neq N^*$; since N^* occupies the same fixed byte offset in both, these strings differ unconditionally whenever $N_i \neq N^*$ — which holds under $\overline{\text{Coll}}$, with no further probabilistic argument needed. By Remark 4.1 (cross-domain distinctness), the tag input also differs unconditionally from every other domain's ρ-input (they differ at byte position 0). Therefore T is a fresh, uniformly distributed 32-byte value independent of b and V.

Consequently, $c^* = (N^*, \tilde{B}, T)$ is jointly uniformly distributed over its domain, independently of b. The adversary gains no information about b from c^*, so $\Pr[\mathcal{A} \text{ wins } G_1 \mid \overline{\text{Coll}}] = \frac{1}{2}$. □

Remark 4.14 (Scope of Lemma 4.12 (audit finding CVF4))**.** *The proof above never inspects the internal structure of B: the OTP argument only uses that* $\text{ks}(N^*)$ *(domain `0x07`) is independent of B, for* any *fixed deterministic B. The prime multiplication $c \cdot k + a$ and the noise tokens assembled into B (domains `0x00`, `0x01`, `0x02`, `0x04`, `0x05`, `0x06`) therefore play no role in this confidentiality argument: per-message content confidentiality (IND-CPA) reduces entirely to domain `0x07` being a secure keyed PRF and to the domain-`0x03` tag for integrity. This does not make the prime/noise layer redundant — it establishes a different, and independent, property; see Lemma 4.15.*

4.2 Traffic-Analysis Resistance (a property independent of Lemma 4.12)

Lemma 4.15 (Ciphertext length is decorrelated from plaintext content)**.** *For any two plaintexts m_0, m_1 of equal padded codepoint length n, encrypted under the same key* $\texttt{kb}$ *and the same nonce N, the ciphertext byte-length $|C| = 16 + 8N_{\text{tok}} + 32$ (where N_{tok} is the real-plus-noise token count) is identical for m_0 and m_1, and depends only on* $(\texttt{kb}, N, n)$ *— never on the codepoint values of m_0 or m_1 — even in a hypothetical world where domain `0x07` is fully distinguishable from random (i.e. this holds independently of Lemma 4.12).*

Proof. The token-construction loop (Section 3.7) advances *ct_pos* by querying $\text{is\_noise}(ct\_pos) = \rho(\texttt{0x00} \| N \| \text{be5}(ct\_pos))$ until n real-token slots have been consumed; this sequence of oracle calls reads only $(\texttt{kb}, N, ct\_pos)$ and never the codepoint value being emitted at a real-token

slot. Consequently N_{tok} is a deterministic function of (kb, N, n) alone, identical for m_0 and m_1 since both have padded length n under the same (kb, N). The fixed-width encoding $|B| = 8N_{\text{tok}}$ (Section 3.7, CVF1 fix) fixes the blob length regardless of token *values*, and domain `0x07` XOR-masks B without changing its length ($|\widetilde{B}| = |B|$). The tag T is a fixed 32-byte HMAC output (Table 1). Hence $|C| = 16 + 8N_{\text{tok}} + 32$ is identical for m_0, m_1 and is a function of (kb, N, n) only, regardless of whether domain `0x07` is secure. □

Remark 4.16 (Scope: this is not a confidentiality claim)**.** *Lemma 4.15 says nothing about whether ciphertext* content *reveals m_0 vs. m_1 — that is Lemma 4.12 and Remark 4.14 above, which depend entirely on domain `0x07` and are indifferent to B's internal structure. Lemma 4.15 instead formalises the noise/prime layer's actual, distinct contribution (audit finding CVF4): ciphertext length does not correlate with plaintext content, a property AES-GCM and ChaCha20-Poly1305 ciphertexts do not have, since their length is an exact linear function of plaintext length. The 2^{197}–2^{257} prime-tuple key-space figures (Table 2) are a key-recovery-resistance statement about the keyed-HMAC construction as a whole, not a statement about what the token structure contributes to either Lemma 4.12 or Lemma 4.15 individually.*

Combining the bounds. By Lemma 4.8, $\Pr[\text{Guess}] \le q_F \cdot 2^{-H_\infty(\mathbf{k})}$, and $\mathcal{A}$ wins G_0 with probability at most 1 whenever Guess occurs, so

$$\Pr[\mathcal{A} \text{ wins } G_0] \;\le\; q_F \cdot 2^{-H_\infty(\mathbf{k})} \;+\; \Pr[\mathcal{A} \text{ wins } G_0 \mid \overline{\text{Guess}}].$$

Conditioned throughout on $\overline{\text{Guess}}$, the G_0/G_1 hop (Lemma 4.9, Remark 4.10) and the hiding lemma proceed exactly as in the original argument:

$$\begin{aligned}\Pr[\mathcal{A} \text{ wins } G_1 \mid \overline{\text{Guess}}] &= \Pr[\text{Coll}] \cdot \Pr[\text{wins} \mid \text{Coll}, \overline{\text{Guess}}] + \Pr[\overline{\text{Coll}}] \cdot \Pr[\text{wins} \mid \overline{\text{Coll}}, \overline{\text{Guess}}] \\ &\le \Pr[\text{Coll}] + \tfrac{1}{2} \;\le\; \frac{q^2}{2^{128}} + \tfrac{1}{2},\end{aligned}$$

so that

$$\left|\Pr[\mathcal{A} \text{ wins } G_0 \mid \overline{\text{Guess}}] - \tfrac{1}{2}\right| \;\le\; \left|\Pr[G_0 \mid \overline{\text{Guess}}] - \Pr[G_1 \mid \overline{\text{Guess}}]\right| + \left|\Pr[\mathcal{A} \text{ wins } G_1 \mid \overline{\text{Guess}}] - \tfrac{1}{2}\right| \;\le\; \text{Adv}_F^{\text{PRF}}(\mathcal{B}_1) +$$

Therefore:

$$\begin{aligned}\text{Adv}^{\text{IND-CPA}}_{\text{NAPQES}}(\mathcal{A}) &= \left|\Pr[\mathcal{A} \text{ wins } G_0] - \tfrac{1}{2}\right| \\ &\le q_F \cdot 2^{-H_\infty(\mathbf{k})} + \left|\Pr[\mathcal{A} \text{ wins } G_0 \mid \overline{\text{Guess}}] - \tfrac{1}{2}\right| \\ &\le \text{Adv}_F^{\text{PRF}}(\mathcal{B}_1) + \frac{q^2}{2^{128}} + q_F \cdot 2^{-H_\infty(\mathbf{k})}.\end{aligned}$$ □

4.3 CVF8: Minimum Key Size and Removing the Residual Non-Standard Assumption

Remark 4.17 (Minimum K for a target security level)**.** *Since $|\mathcal{P}| \gg K$, $H_\infty(\mathbf{k}) = \log_2 |\mathcal{P}|!/(|\mathcal{P}| - K)! \approx K \log_2 |\mathcal{P}| \approx 19.16\,K$ bits for $|\mathcal{P}| \approx 586{,}000$. Theorem 4.7's key-guessing term $q_F \cdot 2^{-H_\infty(\mathbf{k})}$ is negligible against the paper's own ≈ 128-bit post-Grover target (Section 1) only if $H_\infty(\mathbf{k}) \gtrsim 128$, i.e. $K \ge \lceil 128/19.16 \rceil = 7$. $K = 1$–6 ($H_\infty \approx 19$–115 bits) are* not *admissible under this target: for $K = 1$ in particular, $q_F \approx 2^{20}$ offline HMAC evaluations make the guessing term ≈ 1, exactly the brute-force attack Remark 4.2 describes. The paper's default $K = 10$ ($H_\infty \approx 196$ bits) exceeds this minimum with margin. Implementations and future revisions of this specification MUST enforce $K \ge 7$ (with the current $|\mathcal{P}|$) and SHOULD retain $K = 10$ or larger; $K < 7$ should be treated as a distinct, lower security level, not as "IND-CPA secure" in the sense of Theorem 4.7.*

Remark 4.18 (HMAC key-length handling)**.** kb *has length* $5K$ *bytes. HMAC-SHA256 hashes any key longer than its 64-byte block size down to 32 bytes before use, and zero-pads any shorter key up to 64 bytes; neither transformation is addressed by the derivation in Section 3. For* $K \leq 12$ *(*$5K \leq 60$ *bytes),* kb *is used directly, zero-padded to 64 bytes, and Theorem 4.7 applies to* kb *as defined. For* $K \geq 13$ *(*$5K \geq 65$ *bytes, outside the paper's default), the effective HMAC key becomes* $\mathrm{SHA256}(\mathsf{kb})$*: since* $\mathsf{kb} \mapsto \mathrm{SHA256}(\mathsf{kb})$ *is collision-resistant and its domain here (*$2^{H_\infty(\mathbf{k})}$ *possible* kb *values) is far smaller than* 2^{256}*, this map is injective with overwhelming probability and does not reduce* $H_\infty(\mathbf{k})$ *under the PRF-under-D assumption of Remark 4.10; Theorem 4.7 continues to apply with* kb *read as* $\mathrm{SHA256}(\mathsf{key\_bytes}(\mathbf{k}))$ *in that regime. Either way, the theorem's guessing term is unaffected, since an adversary can still evaluate the same key-length-handling transformation offline for each guess at unit cost.*

Remark 4.19 (Recommendation: deriving a uniform subkey removes the non-standard assumption)**.** *Remark 4.10 left Theorem 4.7 resting on* $\mathrm{Adv}^{\mathrm{PRF}}_{\text{HMAC-SHA256}}(\mathcal{B}_1)$ *being negligible* for the specific structured distribution D*, not for a uniform key — a strictly weaker and non-standard assumption relative to the literature's usual uniform-key HMAC-SHA256 PRF conjecture. This residual assumption can be removed entirely, restoring the standard uniform-key PRF game, by deriving a uniformly-distributed HMAC subkey from* kb *via a KDF before use, e.g.* $\mathsf{sk} = \text{HKDF-SHA256}(\mathsf{salt}, \mathsf{kb}, \texttt{"napqes-hmac-key"})$*, and keying every* Derive_d *call with* sk *in place of* kb*. Modelling the KDF as a random oracle (standard for HKDF extract-then-expand),* sk *is statistically close to uniform over* $\{0,1\}^{256}$ *regardless of D's structure, so* $\mathrm{Adv}^{\mathrm{PRF}}_{\text{HMAC-SHA256}}(\mathcal{B}_1)$ *in a restated Theorem 4.7 would then legitimately mean the standard, uniform-key advantage, at the cost of one extra HKDF evaluation per key and a wire-format change (this KDF step is not present in the current reference implementations* `napqes.py/rust/src/lib.rs/C/napqes.c`*). We record this as the preferred long-term fix but have* not *shipped it as part of the CVF8 response, since it changes the wire format and key schedule and is out of scope for a proof-only correction; until it lands, Theorem 4.7 as restated above (with the explicit* $q_F \cdot 2^{-H_\infty(\mathbf{k})}$ *term, Remark 4.10's PRF-under-D caveat, and Remark 4.17's* $K \geq 7$ *floor) is the accurate statement of what is actually proved.*

Remark 4.20 (Erratum: HKDF-over-kb does not, by itself, close CVF13)**.** *The paragraph above additionally claimed that deriving* $\mathsf{sk} = \mathrm{HKDF}(\mathsf{kb})$ *resolves the reduction-simulation gap of CVF13 (Remark 4.11) "once* sk *is independent of* $\mathbf{k}$*." This is imprecise and, on reflection, incorrect as a fix for CVF13 specifically:* $\mathsf{sk} = \mathrm{HKDF}(\mathsf{kb})$ *is a* deterministic function *of* $\mathbf{k}$ *(via* $\mathsf{kb} = \mathsf{key\_bytes}(\mathbf{k})$*), so it is not statistically or computationally independent of* $\mathbf{k}$ *in the sense the simulation argument needs — a reduction that already knows* $\mathbf{k}$ *(required to run the arithmetic layer) can compute* kb *and thus* sk *itself, so forwarding domain-derivation calls to an external oracle keyed by* $\mathsf{sk} = \mathrm{HKDF}(\mathsf{kb})$ *would be vacuous: the reduction never needs the oracle in the first place. The leftover-hash-style argument sketched above only shows* sk*'s* marginal *distribution is close to uniform, which is the right notion for CVF8 (key-entropy) but the wrong notion for CVF13 (simulatability), which requires* sk *to be sampled* independently of $\mathbf{k}$*, not merely to look uniform when* $\mathbf{k}$ *is also known. Closing CVF13 therefore requires* KeyGen *to sample* sk *via a fresh, independent CSPRNG draw — never as any deterministic function of* $\mathbf{k}$ *or* kb*, HKDF included. Section 3.11 below specifies and Section 7 implements exactly this:* KeyGen_{v8} *samples* $\mathbf{k}$ *and* sk *independently, closing both CVF8 (Section 3.11, uniform* sk*) and CVF13 (Section 3.11, genuinely simulatable reductions) simultaneously, for callers who adopt the v8 schedule.*

4.4 INT-CTXT: Ciphertext Integrity

Remark 4.21 (Audit finding CVF10: the bound omitted the ideal-world tag-guessing term and was false for $q = 0$)**.** *The original statement of Theorem 4.24 bounded* $\mathrm{Adv}^{\text{INT-CTXT}}_{\text{NAPQES}}(\mathcal{A})$ *by* $\mathrm{Adv}^{\mathrm{PRF}}_{\text{HMAC-SHA256}}(\mathcal{B}_2) + q^2/2^{128}$ *alone. This is unsound: the reduction* $\mathcal{B}_2$*'s advantage is, by definition,* $\Pr[\textit{forge} \mid \textit{real}] - \Pr[\textit{forge} \mid \textit{random}]$*, so a correct bound on* $\Pr[\textit{forge} \mid \textit{real}]$ *must carry the ideal-world term* $\Pr[\textit{forge} \mid \textit{random}]$ *forward, not discard it — yet even against a* truly random

function, an adversary that submits a candidate tag T^* for a fresh (never-queried) input succeeds with probability exactly 2^{-256} per attempt, purely by guessing. The original statement's omission of this term made the bound literally false at $q = 0$: an adversary making no *encryption queries and submitting a single uniformly random 256-bit tag as its forgery still wins the INT-CTXT game with probability $2^{-256} > 0$, strictly exceeding the original claimed bound of 0 at $q = 0$. The original proof also never stated $\mathcal{B}_2$'s distinguishing rule, so no advantage accounting connected $\mathcal{A}$'s forgery to $\mathcal{B}_2$'s output at all. The restated theorem below adds the missing $q_v/2^{256}$ term, where q_v is the number of forgery-submission (verification) attempts $\mathcal{A}$ makes, and the proof states $\mathcal{B}_2$'s decision rule explicitly (guess "real" iff $\mathcal{A}$'s forgery verifies). This omission propagated into the IND-CCA bound (Theorem 4.30), which invokes this theorem with q_d verification queries and must therefore carry a matching $q_d/2^{256}$ term, added below.*

Remark 4.22 (Audit finding CVF11: Case 2's "nonce collision" term treated an adversarially–chosen value as a random event)**.** *The restated Theorem 4.24 above (post-CVF10) still carried a term $q^2/2^{128}$ into Case 2, justified by the sentence "the probability of a nonce collision satisfying $N^* = N_i$ for some i is at most $q/2^{128} \le q^2/2^{128}$ by the union bound." This is unsound: N^* is not sampled by any experiment. It is a component of the forgery c^* chosen by $\mathcal{A}$ itself, adaptively, after observing $N_1, \ldots, N_q$ in the returned ciphertexts. $\mathcal{A}$ can therefore set $N^* = N_i$ for any i of its choosing with probability* 1*; this is not a random event over which a union bound applies, and no such bound may be used to argue $\Pr[N^* = N_i]$ is small. The error is moreover unnecessary: Case 2's own argument already disposes of $N^* = N_i$* deterministically*, via the injectivity of the domain-`0x03` tag-input encoding (either $\widetilde{B}^* \neq \widetilde{B}_i$, reducing to Case 1, or $\widetilde{B}^* = \widetilde{B}_i$, in which case the tag input is identical to a query-phase input and any $T^* \neq T_i$ is rejected outright) — so no probability term is needed for Case 2 at all, and the $q^2/2^{128}$ term is removed below. The propagation of this term into the IND-CCA bound (Theorem 4.30, via "$q + q_d$ total oracle queries") is removed for the same reason; the genuine $q^2/2^{128}$ term that remains in Theorem 4.30 comes only from the honestly-random challenge nonce in the IND-CPA hybrid (Lemma 4.12's* Coll *event), not from INT-CTXT.*

Remark 4.23 (Audit finding CVF12: stale pre-CVF2 tag-input formula survived in Theorem 4.24's Case 1 and the AAD Binding paragraph)**.** *Theorem 4.24's Case 1 argument and the AAD Binding paragraph below its proof both wrote the tag input as `0x03`$\|\mathrm{be4}(|aad|)\|aad\|N\|\widetilde{B}$ — the* pre*-CVF2, AAD-first-then-nonce layout, in which the nonce sits at a variable byte offset depending on $|aad|$ — even though Remark 4.1 (the CVF2 fix) and the hiding lemma's "Freshness of the authentication tag" paragraph (Lemma 4.12) already state, correctly, that domain `0x03` uses the unified $d\|N\|$`ctx` layout with the nonce at the fixed offset* 1..16*, matching the shipped code (`napqes.py`'s `_compute_auth_tag`, and the equivalent Rust/C routines). This is a real internal inconsistency: the same construction was described by two different, mutually incompatible formulas in the same document, and the stale formula is precisely the one under which the audit's cross-group collision scenario is possible — a nonce-first input from another domain, $N_i\|d\|$`ctx` (21–22 bytes for the fixed-width-counter domains), can equal an AAD-first tag input `0x03`$\|\mathrm{be4}(0)\|N^*$ (21 bytes, the empty-AAD, $|\widetilde{B}^*| = 0$ minimal case) whenever N_i happens to begin with `0x03 00 00 00 00`, an event of probability $\approx 2^{-40}$ per query (or $\approx q/2^{40}$ over q queries by the union bound) — exactly the term the audit finding identifies as missing under that (stale) formula. Under the* actual*, already-unified domain-first layout, this collision cannot occur at all: Remark 4.1's cross-domain argument (inputs with different domain bytes differ at byte position* 0*, unconditionally) already rules out any `0x03` input colliding with a nonce-first domain's input, with no probabilistic term of any kind. The fix below corrects Theorem 4.24's Case 1 and the AAD Binding paragraph to the formula Remark 4.1 and the hiding lemma already use, and grounds Case 1's freshness argument in injectivity (per Remark 4.1) rather than the previously imprecise "prefixed by nonces" phrasing — matching the audit's own recommended fix for the domain-first case. We also rechecked Remark 4.1 for the audit's separately-flagged claim that a $q/2^{32}$ cross-group term is "dominated by" the nonce-collision term $q^2/2^{128}$ (mathematically false*

for realistic q, as the finding correctly notes); no such claim is present anywhere in the current Remark 4.1 — the CVF2 rewrite already replaced the probabilistic cross-group argument with the unconditional injectivity argument, so there is no $q/2^{32}$ term left to (mis)dominate, and no bound in this document ever carried one. No further fix is required for that sub-claim beyond confirming its absence.

Theorem 4.24 (INT-CTXT)**.** *Under the PRF assumption on HMAC-SHA256, NAPQES is INT-CTXT secure. Specifically, for any PPT adversary $\mathcal{A}$ making at most q encryption queries and at most $q_v \geq 1$ forgery-submission (verification) attempts, there exists a PRF adversary $\mathcal{B}_2$ with similar running time such that:*

$$\mathrm{Adv}^{\text{INT-CTXT}}_{\text{NAPQES}}(\mathcal{A}) \leq \mathrm{Adv}^{\text{PRF}}_{\text{HMAC-SHA256}}(\mathcal{B}_2) + \frac{q_v}{2^{256}}.$$

For a single forgery attempt ($q_v = 1$, the minimal INT-CTXT experiment), the last term is 2^{-256}; it is what makes the bound correctly nonzero at $q = 0$ (Remark 4.21). There is no q-dependent term: q (the number of encryption queries) affects only which nonces $\mathcal{A}$ has seen, and Case 2 below disposes of $N^* = N_i$ deterministically rather than probabilistically (Remark 4.22). The experiment grants $\mathcal{A}$ a genuine encryption oracle and a genuine, adaptive *decryption/verification oracle: q_v counts real-time oracle queries, each answered with the true decrypted plaintext or $\perp$ before $\mathcal{A}$ chooses its next query, not a batch of final guesses evaluated only once at the end (Remark 4.26).*

Remark 4.25 (Audit finding CVF20: Case 2 ignored aad^*, and freshness was defined on c^* alone)**.** *An earlier draft of this proof (a) defined freshness as $c^* \notin \{c_1, \ldots, c_q\}$ — a condition on the ciphertext alone — and (b) split Case 2 into $\widetilde{B}^* \neq \widetilde{B}_i$ versus $\widetilde{B}^* = \widetilde{B}_i \wedge T^* \neq T_i$, asserting in the second sub-case that "the tag input is identical to x_i." That claim holds only if additionally $aad^* = aad_i$, which was never assumed: if $aad^* \neq aad_i$ while $\widetilde{B}^* = \widetilde{B}_i$, then $x^* \neq x_i$ (Table 1's length-prefixed* be4$(|aad|)\|aad$ *encoding is injective in aad), and this sub-case must route to the Case 1 fresh-input argument rather than being dismissed as unable to contribute. The standard INT-CTXT object of freshness is the pair (c, aad), not c alone: a forgery that replays c_i verbatim with a new $aad^* \neq aad_i$ is a pair never returned by the encryption oracle, hence a legitimate forgery attempt, and (a) silently excluded it rather than proving it infeasible. The fix below (i) restates freshness over (c^*, aad^*) and (ii) re-splits Case 2 on $x^* = x_i$ versus $x^* \neq x_i$, which subsumes and correctly routes the AAD-replay case.*

Remark 4.26 (Audit finding CVF21: Theorem 4.24 must be proved in the adaptive, multi--verification-query oracle regime the IND-CCA proof actually invokes it in)**.** *Theorem 4.24's statement described q_v only as a count of "forgery-submission attempts," which reads as a batch of final guesses evaluated once at the end of the experiment. The IND-CCA proof (Theorem 4.30), however, invokes this theorem to bound* $\Pr[D]$ *for a forger $\mathcal{A}'$ that holds a genuine, real-time decryption oracle $\hat{\mathcal{D}}$ and answers each of $\mathcal{A}'$s q_d decryption queries adaptively, with $\mathcal{A}$ observing each $\perp$/plaintext answer before choosing its next query — a strictly stronger, Bellare–Namprempre-style multi-query oracle regime that the original statement, read literally, never established. Two further gaps compounded this, per the finding: (i) an earlier draft's Case 2 carried a $q^2/2^{128}$ term into the IND-CCA bound via "$q + q_d$ total oracle queries" (Remark 4.22), substituting $q + q_d$ for the encryption-query parameter q of a term that was, in any case, a collision bound over* encryption-generated *nonces — decryption queries generate no nonces, so that substitution had no basis in the theorem being invoked, independently of the term's own soundness; and (ii) the forger $\mathcal{A}'$ constructed in Theorem 4.30's proof must itself produce the IND-CCA challenge ciphertext c^* (by sampling b and querying its own encryption oracle on m_b) to run the post-challenge phase of $\mathcal{A}$ at all — $q + 1$ encryption queries, not q — yet an earlier draft's simulation description omitted this challenge-generation step entirely while asserting, rather than showing, that "$\mathcal{A}'$s simulation is perfect."*

Resolution. *We resolve this by proving the bound holds in exactly the adaptive oracle regime required, rather than restating it as a weaker, single-shot game:*

- Adaptivity does not change the Case 1/Case 2 argument. *Fix any sequence of q_v adaptive decryption/verification queries, each answered in real time before the next is chosen. Case 2 (above) is resolved by injectivity alone — for a query with $N = N_i$ for some encryption-query i, whether $x = x_i$ or $x \neq x_i$ is fully determined by the query's own bytes, independent of any other query's answer or ordering, so adaptivity contributes nothing to Case 2 regardless of how many queries have already been answered. Case 1 (a query with a fresh N) reduces the query's success probability, in the ideal world, to $f(x)$ for an x that — by the same injectivity argument — has never been queried to f before,* even counting every previous decryption/verification query's own x as part of f's query history*: lazy sampling of the random function f gives every not-yet-queried input an independent uniform 256-bit value regardless of what other, distinct inputs have already been resolved, so knowledge of every previous query's outcome (accept or $\perp$) reveals nothing about f at a new, still-fresh point. The per-query success probability is therefore exactly 2^{-256} regardless of adaptivity, position in the query sequence, or $\mathcal{A}$'s strategy for choosing later queries from earlier answers, and the union bound* $\Pr[$*forge in Case 1 across all q_v queries*$] \leq q_v/2^{256}$ *therefore holds for the fully adaptive, real-time oracle exactly as it did for a final batch of guesses — the* bound *of Theorem 4.24 is unchanged, but it is now established for the stronger regime the IND-CCA proof requires, closing this half of the finding.*
- No substitution of $q + q_d$ into a nonce-collision parameter is needed*, because none exists to substitute into: Remark 4.22 already removed Case 2's $q^2/2^{128}$ term from Theorem 4.24 entirely (it traced to a bogus argument over the adversarially-chosen N^*, not to a genuine collision over encryption-generated nonces), so Theorem 4.24's bound has no q-dependent term at all for the IND-CCA proof to (mis)parametrise by $q + q_d$. This was gap (i) above; it is moot under the current bound rather than requiring a corrected substitution.*
- The forger's own challenge query, and the perfect-simulation claim, are made explicit *in the revised construction of $\mathcal{A}'$ below (Theorem 4.30's proof): $\mathcal{A}'$ is shown, step by step, to sample b itself and query its own encryption oracle on m_b as its $(q{+}1)$-th encryption query, to record the resulting c^* for later freshness bookkeeping (Remark 4.29), and to forward every one of $\mathcal{A}$'s decryption queries — pre-* and *post-challenge (Remark 4.28) — to its own real decryption oracle; perfection of the simulation is then derived from the fact that every oracle $\mathcal{A}$ interacts with is a real oracle keyed by the same hidden key throughout, rather than merely asserted.*

Proof. The INT-CTXT experiment requires $\mathcal{A}$ to produce a *fresh* valid ciphertext: a pair (c^*, aad^*) with $(c^*, aad^*) \notin \{(c_1, aad_1), \ldots, (c_q, aad_q)\}$ (the outputs of the q encryption oracle calls, paired with their queried AAD; Remark 4.25) such that $\text{NAPQES.}\mathsf{Dec}(\mathbf{k}, c^*, aad^*) \neq \perp$. In particular $\mathcal{A}$ may submit $c^* = c_i$ for some i provided $aad^* \neq aad_i$ — a replay-with-new-AAD attempt, addressed in Case 2 below.

Write $c^* = (N^*, \widetilde{B}^*, T^*)$. For Dec to accept, T^* must satisfy (domain-first layout, Remark 4.1; corrected here per audit finding CVF12, Remark 4.23 — an earlier draft of this proof used the pre-CVF2, AAD-first-then-nonce formula):

$$T^* = \text{HMAC}\Big(\mathsf{kb},\ \mathtt{0x03}\|N^*\|\text{be4}(|aad^*|)\|aad^*\|\widetilde{B}^*\Big)\,.$$

Let $x^* = \mathtt{0x03}\|N^*\|\text{be4}(|aad^*|)\|aad^*\|\widetilde{B}^*$.

Case 1: N^* was not used in any encryption query. Every domain-`0x03` tag input from the query phase has the form $x_i = \mathtt{0x03}\|N_i\|\text{be4}(|aad_i|)\|aad_i\|\widetilde{B}_i$, with N_i occupying the same fixed byte offset 1..16 that N^* occupies in x^* (Remark 4.1: every domain-`0x03` input shares the uniform $d\|N\|\mathsf{ctx}$ shape). Since $N_i \neq N^*$ for every query i (Case 1 hypothesis), $x_i \neq x^*$ unconditionally

at bytes 1..16, regardless of $|aad_i|$, $|aad^*|$, or $\widetilde{B}_i$ — freshness follows from injectivity of the encoding, not from an assumed nonce prefix, and no probabilistic cross-group collision term (of the kind audit finding CVF12 checked for; see Remark 4.23) is needed or applies: Remark 4.1's cross-domain distinctness additionally rules out x^* colliding with any other domain's input. So x^* was never submitted to HMAC($\mathsf{kb}, \cdot$) during the query phase. We construct $\mathcal{B}_2$, given oracle access to a function f that is either HMAC($\mathsf{kb}, \cdot$) (the *real* world) or a uniformly random function $\{0,1\}^* \to \{0,1\}^{256}$ (the *ideal* world): $\mathcal{B}_2$ simulates the INT-CTXT experiment for $\mathcal{A}$, forwarding every HMAC computation to f, and evaluates each of $\mathcal{A}$'s (at most q_v) forgery-submission attempts by computing the expected tag via f. This description shares, for the same reason, the simulation gap identified in Remark 4.11: answering $\mathcal{A}$'s q encryption queries requires running NAPQES.$\mathsf{Enc}(\mathbf{k}, \cdot)$, which needs $\mathbf{k}$ itself for its arithmetic layer, not merely oracle access to f; see that remark for the resolution. *Decision rule:* $\mathcal{B}_2$ outputs "real" iff at least one of $\mathcal{A}$'s forgeries verifies (its candidate tag matches $f(x^*)$ for the corresponding tag input x^*); otherwise it outputs "random". By this construction,

$$\mathrm{Adv}^{\mathrm{PRF}}_{\text{HMAC-SHA256}}(\mathcal{B}_2) = \Pr[\mathcal{B}_2 \to \text{"real"} \mid f = \mathrm{HMAC}(\mathsf{kb}, \cdot)] - \Pr[\mathcal{B}_2 \to \text{"real"} \mid f \text{ random}] = \Pr[\text{forge} \mid \text{real}] - \Pr[$$

In the real world this probability is exactly $\mathcal{A}$'s Case-1 forgery probability. In the ideal world, x^* was never queried, so $f(x^*)$ is a uniform 256-bit string independent of $\mathcal{A}$'s view; each of $\mathcal{A}$'s q_v forgery attempts therefore succeeds only by blind guessing, with probability exactly 2^{-256} per attempt, so by the union bound $\Pr[\text{forge} \mid \text{random}] \le q_v/2^{256}$. Rearranging the advantage equation,

$$\Pr[\text{forge in Case 1}] \;\le\; \mathrm{Adv}^{\mathrm{PRF}}_{\text{HMAC-SHA256}}(\mathcal{B}_2) + \frac{q_v}{2^{256}}.$$

This ideal-world term is exactly what the original proof omitted (Remark 4.21): it is what makes the bound correct at $q = 0$, since an adversary submitting one uninformed random tag guess still forges with probability 2^{-256} even against a perfect random function.

Case 2: $N^* = N_i$ for some query i. Fix any query i with $N_i = N^*$, and recall $x_i = \mathsf{0x03} \| N_i \| \mathrm{be4}(|aad_i|) \| aad_i \| \widetilde{B}_i$. The correct split (Remark 4.25) is on $x^* = x_i$ versus $x^* \neq x_i$, not on $\widetilde{B}^*$ alone:

- *$x^* \neq x_i$ for every query i with $N_i = N^*$.* This covers $\widetilde{B}^* \neq \widetilde{B}_i$ *and* the case $\widetilde{B}^* = \widetilde{B}_i$ with $aad^* \neq aad_i$ (by injectivity of $\mathrm{be4}(|aad|) \| aad$ in aad, holding $N_i = N^*$ and $\widetilde{B}^* = \widetilde{B}_i$ fixed) — in particular the replay-with-new-AAD attempt $(c^*, aad^*) = (c_i, aad^*)$, $aad^* \neq aad_i$, falls here. In every such case x^* was never submitted to HMAC($\mathsf{kb}, \cdot$) during the query phase (it differs from every x_j: from x_i by the argument just given, and from every x_j with $N_j \neq N^*$ by the Case 1 argument applied to that j), so the Case 1 reduction and bound apply verbatim to this forgery attempt.
- *$x^* = x_i$ for some query i with $N_i = N^*$.* By injectivity of the tag-input encoding (Remark 4.1), $x^* = x_i$ forces $aad^* = aad_i$ and $\widetilde{B}^* = \widetilde{B}_i$ jointly. Two sub-cases on the tag itself: if $T^* = T_i$, then $c^* = (N^*, \widetilde{B}^*, T^*) = c_i$ and $aad^* = aad_i$, so $(c^*, aad^*) = (c_i, aad_i)$ — excluded by the freshness requirement (Remark 4.25), hence not a valid forgery attempt. If instead $T^* \neq T_i$, decryption recomputes the expected tag as $\mathrm{HMAC}(\mathsf{kb}, x^*) = \mathrm{HMAC}(\mathsf{kb}, x_i) = T_i \neq T^*$ and rejects outright, regardless of freshness. Either way this branch contributes no successful, freshness-respecting forgery.

Both branches are exhaustive and are resolved *deterministically* by injectivity, not by any probabilistic argument over N^*: N^* is a value $\mathcal{A}$ chooses as part of its forgery c^*, adaptively and after observing $N_1, \ldots, N_q$, not a value sampled by the experiment, so no probability term (in particular, no $q^2/2^{128}$ nonce-collision term) is associated with Case 2 (Remark 4.22). The re-split above additionally shows the replay-with-new-AAD attempt is not silently excluded but is proved infeasible beyond the Case 1 bound: it always falls in the first branch, which reduces

to Case 1. Case 2 therefore contributes no additional term beyond Case 1's, and combining both cases:

$$\mathrm{Adv}^{\text{INT-CTXT}}_{\text{NAPQES}}(\mathcal{A}) \;\le\; \mathrm{Adv}^{\text{PRF}}_{\text{HMAC-SHA256}}(\mathcal{B}_2) + \frac{q_v}{2^{256}}. \qquad \square$$

4.5 AAD Binding

The AAD is included in the tag computation as (domain-first layout, Remark 4.1; corrected here per audit finding CVF12, Remark 4.23): $T = \mathrm{HMAC}(\mathtt{kb},\ \mathtt{0x03} \| N \| \mathrm{be4}(|A|) \| A \| B)$. The 4-byte length prefix prevents length-extension mixing between different AAD lengths. Any single-bit change to the AAD produces an unpredictable change to the expected tag, with advantage bounded by the HMAC security.

4.6 IND-CCA Security

Remark 4.27 (Audit finding CVF14: the $H_0 \to H_1$ "identical-until-bad" step is false, since H_1 also diverges on non-fresh replays)**.** *An earlier draft of the proof below defined H_1 by stubbing $\mathcal{D}$ to* always *return $\perp$, while defining the bad event D as a* fresh *($c \notin \{c_1, \ldots, c_q\}$) valid submission, and then asserted "H_0 and H_1 are identical in every execution that does not trigger D." That assertion is false as those two definitions stand: in H_0, $\mathcal{A}$ may legally query $\mathcal{D}$ on a* replayed *encryption-oracle output (c_i, aad_i) and receive the correct plaintext m_i (by correctness, Definition 3.18), whereas the stubbed H_1 returns $\perp$ on that same query — yet this query is not fresh, so D does not occur. An adversary that simply queries $\mathcal{D}(c_1, aad_1)$ after one encryption-oracle query distinguishes H_0 from H_1 with probability* 1 *without ever triggering D, so the fundamental-lemma step $|\Pr[H_0] - \Pr[H_1]| \le \Pr[D]$ was not established as written. This is a distinct defect from CVF10–CVF13, which concern gaps or omitted terms inside the INT-CTXT and IND-CCA advantage bounds themselves; CVF14 concerns whether the hybrid argument used to* reduce *IND-CCA to INT-CTXT plus IND-CPA is even correctly stated, independent of those bounds' content. The fix, applied below, is the standard corrected hybrid: redefine H_1 so that $\mathcal{D}$ answers a replayed (c_i, aad_i) pair with its recorded plaintext m_i from a table populated by the encryption oracle, and returns $\perp$ only for ciphertexts not in that table (i.e., only for fresh queries). Under that definition H_0 and H_1 agree exactly on replayed queries (both return m_i, by correctness) and can only disagree on a fresh query, which is exactly what D captures, so the identical-until-bad step now goes through.*

Remark 4.28 (Audit finding CVF22: the bad event D was restricted to the query phase, leaving post-challenge forgeries uncovered)**.** *Game H_0 below explicitly permits $\mathcal{A}$ to call the decryption oracle after receiving the challenge ciphertext, yet an earlier draft defined the bad event D as a fresh valid submission occurring only "during the query phase" (i.e., before the challenge). A fresh valid ciphertext submitted* after *the challenge makes H_0 and H_1 diverge (by exactly the same case analysis as a pre-challenge query) but was not counted by that restricted D, so the identical-until-bad bound $|\Pr[H_0] - \Pr[H_1]| \le \Pr[D]$ again failed to cover every distinguishing execution — the same defect family as CVF14, applied to the phase of the query rather than to replays.* ***Fix:*** *D below is defined over decryption queries in* both *phases (any fresh valid (c, aad) submitted to $\mathcal{D}$ at any point in the experiment), and the INT-CTXT forger $\mathcal{A}'$ constructed in the proof forwards* every *one of $\mathcal{A}$'s decryption queries — pre- and post-challenge alike — to its own real decryption oracle, so the reduction bounding $\Pr[D]$ runs across both phases exactly as D is now defined.*

Remark 4.29 (Audit finding CVF23: freshness-set mismatch, and c-only freshness silently weakened the IND-CCA notion)**.** *Two related defects. First, an earlier draft's bad event D used the freshness set $\{c_1, \ldots, c_q\}$ (the adversary's own q encryption queries), but from the INT-CTXT challenger's point of view the challenge ciphertext c^* is itself an $\hat{\mathcal{E}}$ output once the forger $\mathcal{A}'$ generates it (Remark 4.26), so the correct freshness set for $\mathcal{A}'$'s win condition is $\{c_1, \ldots, c_q\} \cup \{c^*\}$; the two sets were never reconciled. Second, and independently, an earlier*

draft's Game H_0 forbade $\mathcal{A}$ from querying $\mathcal{D}$ on any $c = c^*$ *(for every aad), whereas the standard AEAD IND-CCA game forbids only the specific pair (c^*, aad^*) and* permits $\mathcal{D}(c^*, aad \neq aad^*)$ *— a legitimate, potentially-winning forgery attempt in the standard game that the c-only restriction silently excluded, consistent with the c-only (rather than (c, aad)-pair) freshness already corrected for Theorem 4.24 by audit finding CVF20 (Remark 4.25). Left uncorrected, the composed result would establish a strictly weaker-than-standard IND-CCA notion — one in which $\mathcal{A}$ is never even permitted to* attempt *the AAD-substitution forgery, rather than one in which that forgery is proved to fail.* ***Fix:*** *the table T used by D/D' below is populated to include the challenge tuple (c^*, aad^*, m_b) once the challenge is issued, so $\mathcal{A}'$'s freshness set is exactly $\{(c_1, aad_1), \ldots, (c_q, aad_q)\} \cup \{(c^*, aad^*)\}$ — the INT-CTXT challenger's own complete output set, reconciling the mismatch — and Game H_0 now forbids only the pair (c^*, aad^*), explicitly permitting $\mathcal{D}(c^*, aad \neq aad^*)$, so the proven notion is the standard (c, aad)-pair-freshness AEAD IND-CCA game rather than the c-only weakening.*

Theorem 4.30 (IND-CCA)**.** *Under the PRF assumption on HMAC-SHA256, NAPQES is IND-CCA secure. Specifically, for any PPT adversary $\mathcal{A}$ making at most q encryption queries and q_d decryption queries there exist PRF adversaries $\mathcal{B}_1$, $\mathcal{B}_2$ with similar running time such that:*

$$\mathrm{Adv}^{\text{IND-CCA}}_{\text{NAPQES}}(\mathcal{A}) \leq \mathrm{Adv}^{\text{PRF}}_{\text{HMAC-SHA256}}(\mathcal{B}_1) + \mathrm{Adv}^{\text{PRF}}_{\text{HMAC-SHA256}}(\mathcal{B}_2) + \frac{q^2}{2^{128}} + \frac{q_d}{2^{256}}.$$

The $q_d/2^{256}$ term is the ideal-world forgery term of Theorem 4.24 (Remark 4.21), carried through with $q_v = q_d$ since the INT-CTXT forger $\mathcal{A}'$ constructed below makes exactly q_d verification (decryption-oracle) queries, counted across both *the pre- and post-challenge phases (Remark 4.28). Freshness for D and for the INT-CTXT reduction is measured over the pair (c, aad) against the set $\{(c_1, aad_1), \ldots, (c_q, aad_q)\} \cup \{(c^*, aad^*)\}$ (Remark 4.29). The forger $\mathcal{A}'$ constructed below additionally uses $q+1$ total encryption-oracle queries (its q forwarded queries plus one to produce c^* itself) when invoking Theorem 4.24 — which carries no encryption-query-count-dependent term (Remark 4.22) and is proved in the adaptive dual-oracle regime this invocation requires (Remark 4.26) — so this does not change the stated bound. The $q^2/2^{128}$ term is the genuine nonce-collision bound from the IND-CPA hybrid (Lemma 4.12's* Coll *event over the q honestly-random encryption-oracle nonces plus the challenge nonce); it is* not *inherited from Theorem 4.24, which has no q-dependent term (Remark 4.22). (As noted in Remark 4.27, the hybrid game H_1 used in the proof below is defined with a table-backed decryption oracle rather than one stubbed unconditionally to $\perp$, so that the identical-until-bad step is sound.)*

Proof. We apply the composition theorem of Bellare and Namprempre [13] (B&N 2000, Theorem 3): a scheme that is simultaneously IND-CPA and INT-CTXT is IND-CCA. We make the argument explicit via two hybrid games. (As noted in Remark 4.11, the $\mathrm{Adv}^{\text{PRF}}(\mathcal{B}_1)$ and $\mathrm{Adv}^{\text{PRF}}(\mathcal{B}_2)$ terms this composition invokes below inherit that remark's simulation-gap caveat.)

Game H_0 (Real IND-CCA). The standard IND-CCA experiment: $\mathcal{A}$ has access to an encryption oracle $\mathcal{E}$ and a decryption oracle $\mathcal{D}$, and may query $\mathcal{D}$ both before and after receiving the challenge ciphertext c^* (produced by one call to $\mathcal{E}$ on adversarially-chosen (A^*, m_b)). The *only* restriction is on the exact pair: $\mathcal{A}$ may not query $\mathcal{D}(c^*, A^*)$, but *may* query $\mathcal{D}(c^*, aad)$ for any $aad \neq A^*$ (Remark 4.29) — the standard AEAD (c, aad)-pair freshness notion, not the strictly weaker c-only restriction ("$\mathcal{D}$ on any $c \neq c^*$") an earlier draft imposed.

Game H_1 (Table-backed decryption oracle). Identical to H_0, except $\mathcal{D}$ is replaced by $\mathcal{D}'$, which consults a table T populated by every (c, aad, m) triple $\mathcal{E}$ has produced so far — including, once issued, the challenge triple (c^*, A^*, m_b) itself (Remark 4.29: from the standpoint of freshness, the challenge is an $\mathcal{E}$ output like any other): on query (c, aad), $\mathcal{D}'$ returns the recorded m if

(c, aad) matches some entry in T, and returns $\perp$ otherwise — i.e., $\perp$ only on pairs not yet in T (per Remark 4.27; an earlier draft instead stubbed $\mathcal{D}$ to return $\perp$ unconditionally, which is corrected here). The one query the game forbids outright, $\mathcal{D}(c^*, A^*)$, would — were it permitted — trivially hit the table entry for (c^*, A^*, m_b); since $\mathcal{A}$ is never allowed to make it, $\mathcal{D}'$ never needs to disclose m_b to answer any query it actually receives.

Transition $H_0 \to H_1$. Let D be the event that $\mathcal{A}$ submits, *at any point in the experiment — before or after the challenge alike* (Remark 4.28), a *fresh valid* ciphertext to $\mathcal{D}$—i.e., a pair (c, aad) with $(c, aad) \notin T$ (not equal to any (c_i, aad_i) recorded from a pre- or post-challenge encryption-oracle output, nor to (c^*, A^*) itself, Remark 4.29) such that $\text{NAPQES.}\mathsf{Dec}(\mathbf{k}, c, aad) \neq \perp$. We claim H_0 and H_1 answer every decryption query identically unless D occurs, in *either* phase. Consider any query (c, aad) $\mathcal{A}$ makes to the decryption oracle, pre- or post-challenge: (a) if $(c, aad) \in T$, i.e. it replays some earlier $\mathcal{E}$ output (a replay of (c^*, A^*) itself cannot occur, since H_0 forbids that exact query), then by correctness (Definition 3.18) H_0's real decryption returns exactly the recorded plaintext, which is precisely what H_1's table lookup returns — the two games agree on every replayed query, closing exactly the gap identified in Remark 4.27; (b) if $(c, aad) \notin T$ and the ciphertext is invalid, both games return $\perp$; (c) if $(c, aad) \notin T$ and the ciphertext is valid, H_0 returns the (non-$\perp$) decrypted plaintext while H_1 returns $\perp$ — this is exactly event D. This case analysis is identical in the pre-challenge and post-challenge phases — nothing about it depends on whether the challenge has been issued yet, only on whether $(c, aad) \in T$ — so restricting D to a single phase, as an earlier draft did, was an unjustified narrowing (Remark 4.28) rather than a consequence of the case analysis itself. Hence H_0 and H_1 are identical in every execution that does not trigger D in *either* phase, and the fundamental lemma of game-hopping gives:

$$\left|\Pr[\mathcal{A} \text{ wins } H_0] - \Pr[\mathcal{A} \text{ wins } H_1]\right| \;\leq\; \Pr[D].$$

We bound $\Pr[D]$ by constructing an INT-CTXT forger $\mathcal{A}'$ (using the B&N formulation that grants the INT-CTXT adversary both an encryption oracle *and* an adaptive decryption/verification oracle, in the regime proved sound by Remark 4.26 [13], Definition 2):

1. $\mathcal{A}'$ receives an encryption oracle $\hat{\mathcal{E}}$ and a decryption oracle $\hat{\mathcal{D}}$ from the INT-CTXT challenger, and initialises an empty table T.
2. $\mathcal{A}'$ runs $\mathcal{A}_1$, forwarding every encryption query to $\hat{\mathcal{E}}$ (recording each resulting (c_i, aad_i, m_i) triple into T) and every (pre-challenge) decryption query to $\hat{\mathcal{D}}$: whenever $\hat{\mathcal{D}}$ returns a non-$\perp$ result on a pair $(c, aad) \notin T$, event D has occurred and $\mathcal{A}'$ immediately wins the INT-CTXT experiment.
3. When $\mathcal{A}_1$ outputs $(A^*, m_0, m_1, \mathsf{st})$, $\mathcal{A}'$ samples $b \xleftarrow{\$} \{0,1\}$ itself and queries $\hat{\mathcal{E}}$ on (A^*, m_b) — its $(q{+}1)$-th encryption query — to obtain the challenge ciphertext c^*, records (c^*, A^*, m_b) into T (Remark 4.29), and invokes $\mathcal{A}_2(\mathsf{st}, c^*)$: the challenge phase is thus an explicit, counted step of the construction, not an unstated assumption (Remark 4.26, gap (ii)).
4. $\mathcal{A}'$ continues running $\mathcal{A}_2$, forwarding every *post-challenge* decryption query to $\hat{\mathcal{D}}$ exactly as in step 2 — $\mathcal{A}_2$ may query any $(c, aad) \neq (c^*, A^*)$, per Game H_0's restriction (Remark 4.29) — and again declares event D (and wins) on the first non-$\perp$ result for a pair not in T; this step is what makes the reduction cover the post-challenge phase, which an earlier draft's construction omitted (Remark 4.28).
5. If $\mathcal{A}$ terminates without triggering D in either phase, $\mathcal{A}'$ outputs a fixed, known-invalid forgery attempt (e.g. (c^*, A^*) with its tag byte flipped), which fails to verify and so does not win.

Perfect simulation, demonstrated rather than asserted. Every value $\mathcal{A}$ observes in this simulation — every ciphertext returned for an encryption query (including c^*, generated in step 3 by a real call to $\hat{\mathcal{E}}$ under the same hidden challenger key that answers every other query) and every plaintext/$\perp$ returned for a decryption query in either phase (answered by the real oracle $\hat{\mathcal{D}}$ in steps 2 and 4) — is produced by the actual INT-CTXT challenger's real oracles, keyed identically throughout and never diverging from $\mathcal{A}$'s view in H_0 itself; $\mathcal{A}'$ introduces no approximation at any step, so $\mathcal{A}$'s view in this simulation is distributed exactly as in H_0, and event D occurs in the simulation iff it occurs in H_0 (Remark 4.26). Consequently $\Pr[\mathcal{A}' \text{ wins INT-CTXT}] = \Pr[D]$, giving:

$$\Pr[D] \;\le\; \mathrm{Adv}^{\text{INT-CTXT}}_{\text{NAPQES}}(\mathcal{A}') \;\le\; \mathrm{Adv}^{\text{PRF}}_{\text{HMAC-SHA256}}(\mathcal{B}_2) + \frac{q_d}{2^{256}},$$

where the second inequality applies Theorem 4.24 in the adaptive, dual-oracle regime justified by Remark 4.26, with $\mathcal{A}'$ making $q+1$ encryption-oracle queries (contributing no term, since Theorem 4.24 carries no encryption-query-count term, Remark 4.22) and $q_v = q_d$ decryption/verification queries counted across *both* phases (Remark 4.28): $\mathcal{A}'$ submits exactly one forgery candidate per decryption-oracle query $\hat{\mathcal{D}}$ that $\mathcal{A}$ makes, pre- or post-challenge, and none from its $q+1$ encryption queries, so q_d is the correct total count for this term, and the freshness set against which each such query is judged — $T \supseteq \{(c_1, aad_1), \ldots, (c_q, aad_q)\} \cup \{(c^*, A^*)\}$ — is exactly the INT-CTXT challenger's own complete set of $\hat{\mathcal{E}}$ outputs (Remark 4.29), with no mismatch left unreconciled.

Game H_1 reduces to IND-CPA. In H_1, $\mathcal{D}'$ never returns any information $\mathcal{A}$ does not already possess: on a replayed query it returns m_i, the very plaintext $\mathcal{A}$ itself supplied to $\mathcal{E}$ to obtain c_i in the first place, and on any other query it returns $\perp$. (The sole table entry $\mathcal{A}$ could not reconstruct this way is the challenge triple (c^*, A^*, m_b), since $\mathcal{A}''$ below does not know b — but $\mathcal{A}$ is never permitted to query $\mathcal{D}'(c^*, A^*)$ (Remark 4.29), so this entry is never looked up and its absence from $\mathcal{A}''$'s local copy of T never affects an answer $\mathcal{A}$ actually receives.) Hence $\mathcal{D}'$ can be simulated perfectly by $\mathcal{A}$'s own local table T without any oracle access beyond $\mathcal{E}$, so an IND-CPA adversary $\mathcal{A}''$ can reproduce H_1 exactly for $\mathcal{A}$ using only the IND-CPA experiment's encryption oracle (maintaining T itself and answering $\mathcal{A}$'s decryption queries from it), so H_1 is computationally equivalent to the IND-CPA experiment, giving:

$$\Pr[\mathcal{A} \text{ wins } H_1] \;\le\; \tfrac{1}{2} + \mathrm{Adv}^{\text{IND-CPA}}_{\text{NAPQES}}(\mathcal{A}'') \;\le\; \tfrac{1}{2} + \mathrm{Adv}^{\text{PRF}}_{\text{HMAC-SHA256}}(\mathcal{B}_1) + \frac{q^2}{2^{128}},$$

where Theorem 1 is applied to the IND-CPA adversary $\mathcal{A}''$.

Remark 4.31 (Audit finding CVF15: the "absorbing the constant" step in an earlier draft was invalid, since $2x \le x$ is false for $x > 0$)**.** *An earlier draft of the combining step below derived two nonce-collision terms, $q^2/2^{128}$ (from the H_1-reduces-to-IND-CPA step) and $(q+q_d)^2/2^{128}$ (inherited from an earlier, since-retracted version of Theorem 4.24's Case 2 bound), bounded their sum via $q^2 + (q+q_d)^2 \le 2(q+q_d)^2$, and then asserted that "absorbing the constant into the stated bound yields $(q+q_d)^2/2^{128}$, matching the statement." That step is invalid: a multiplicative factor of two cannot be dropped for free, since $2x \le x$ is false for every $x > 0$ — the algebra shown established only $2(q+q_d)^2/2^{128}$, not the bare $(q+q_d)^2/2^{128}$ the theorem claimed.*

This defect no longer applies to the proof as it now stands, for a reason related to but distinct from this finding's own recommendation: Remark 4.22 already showed the $(q+q_d)^2/2^{128}$ term being "absorbed" here was itself unsound — it traced to a bogus union-bound argument over the adversarially-chosen forgery nonce N^* in Theorem 4.24's Case 2 — and removed it from that theorem entirely, rather than merely restating its coefficient. With that term gone, the combining step below sums exactly one genuine nonce-collision term, $q^2/2^{128}$ (Lemma 4.12's Coll *event), with the genuine ideal-world forgery term $q_d/2^{256}$ (Remark 4.21) — a plain addition of two distinct, correctly-scoped terms, with no factor-of-two approximation, tightening, or absorption*

step required at all. Had CVF11 not already eliminated the disputed term, the correct fix would have been exactly this finding's recommendation: state the honest $2(q+q_d)^2/2^{128}$ constant, or tighten via $q^2+(q+q_d)^2 \le (2q+q_d)^2$, rather than silently drop the factor of two.

Combining.

$$\begin{aligned}\mathrm{Adv}^{\text{IND-CCA}}_{\text{NAPQES}}(\mathcal{A}) &= \left|\Pr[\mathcal{A} \text{ wins } H_0] - \tfrac{1}{2}\right| \\ &\le \Pr[D] + \left|\Pr[\mathcal{A} \text{ wins } H_1] - \tfrac{1}{2}\right| \\ &\le \mathrm{Adv}^{\text{PRF}}_{\text{HMAC-SHA256}}(\mathcal{B}_2) + \frac{q_d}{2^{256}} + \mathrm{Adv}^{\text{PRF}}_{\text{HMAC-SHA256}}(\mathcal{B}_1) + \frac{q^2}{2^{128}}.\end{aligned}$$

The single nonce-collision term $q^2/2^{128}$ here originates entirely from the H_1-reduces-to-IND-CPA step (the honestly-random challenge nonce, Lemma 4.12's Coll event); the INT-CTXT bound contributes no such term (Remark 4.22), so no absorption or doubling is needed (Remark 4.31). Carrying the $q_d/2^{256}$ ideal-world forgery term (Remark 4.21) through unchanged yields $\mathrm{Adv}^{\text{IND-CCA}}_{\text{NAPQES}}(\mathcal{A}) \le \mathrm{Adv}^{\text{PRF}}(\mathcal{B}_1) + \mathrm{Adv}^{\text{PRF}}(\mathcal{B}_2) + q^2/2^{128} + q_d/2^{256}$, matching the statement. This combining step and the final bound are unaffected by the CVF14 correction (Remark 4.27) above; only the definition of H_1 and the identical-until-bad argument for $\Pr[D]$ were repaired, not the numerical value of any term. □

5 Post-Quantum Analysis

Remark 5.1 (Audit finding CVF24: the Grover post-quantum figure was stated as a bare number without operational caveats)**.** *The original text stated only that "Grover's algorithm halves the exponent, giving a post-quantum key-search complexity of $\approx 2^{98}$, well above 128-bit thresholds," with no operational context. As the finding notes, a bare exponent invites two misreadings: (i) that 2^{98} is an ordinary, comparably-achievable attack cost, on the same footing as a classical work-factor figure; and (ii), more importantly, that it is a* parallelizable *cost that could be brought within reach by adding hardware. Neither is true. Grover's algorithm requires $\approx 2^{n/2}$* essentially sequential *oracle queries and is well known to be non-parallelizable in any useful sense: the Bennett–Bernstein–Brassard–Vazirani (BBBV) optimality result [5] establishes that no quantum algorithm can search an unstructured space of size N in fewer than $\Omega(\sqrt{N})$ oracle queries, and Zalka's analysis [6] of distributing Grover's search over S independent machines shows the achievable speedup is only $\sqrt{S}$ — the wall-clock* depth *remains $\approx 2^{n/2}/\sqrt{S}$ and the* total *work (query-time × machine-count) is unchanged, in sharp contrast to classical brute force, which parallelizes linearly across machines. Presented as a bare number, 2^{98} therefore looks like a hardware-scalable, classical-style cost rather than the sequential-depth lower bound it actually is. The fix below rephrases both post-quantum figures (the 2^{128} SHA-256 preimage bound and the 2^{98} key-space bound) to state this explicitly.*

SHA-256 and Grover's algorithm. Grover's algorithm [4] provides a quadratic speedup for unstructured search, requiring $\approx 2^{n/2}$ oracle queries against an n-bit target. Applied to SHA-256 preimage search, this reduces the *nominal* security level from 256 bits to approximately 128 bits. Crucially, this 2^{128} figure is a bound on *sequential query depth*, not a parallelizable work-factor: the BBBV optimality result [5] shows no quantum algorithm can search an unstructured space of size N in fewer than $\Omega(\sqrt{N})$ oracle queries, and Zalka [6] shows that distributing Grover's search across S machines yields only a $\sqrt{S}$ speedup — the depth stays $\approx 2^{n/2}/\sqrt{S}$ and the total work is unchanged — so, unlike a classical brute-force search, this cost cannot be brought closer to feasibility by adding hardware. At 2^{128} sequential oracle queries, SHA-256 preimage resistance remains far beyond feasibility and above the 112-bit minimum recommended by NIST for post-2030 systems (Remark 5.1).

No Shor-applicable structure. NAPQES does not use discrete logarithms, integer factorisation, or any algebraic group structure. Shor's algorithm and related quantum algorithms provide no speedup against the construction.

Key space. With a 10-element prime-tuple key from $\mathcal{P}$, the key space is $\approx 2^{196}$. Grover's algorithm halves the exponent, giving a post-quantum key-search complexity of $\approx 2^{98}$ *sequential* oracle queries (Remark 5.1) — like the SHA-256 figure above, this is a non-parallelizable depth bound, not a figure that shrinks with additional hardware: by the same BBBV [5]/ Zalka [6] argument, S machines give only a $\sqrt{S}$ speedup, so the wall-clock depth stays $\approx 2^{98}/\sqrt{S}$ and the total work is unchanged. Both the 2^{128} and 2^{98} figures are therefore sequential-depth quantities far beyond feasibility, and at these output/key-space lengths Grover's algorithm is largely irrelevant to NAPQES's practical post-quantum security margin.

6 Comparison with AES-GCM and ChaCha20-Poly1305

Property	NAPQES	AES-GCM	ChaCha20-Poly1305	Ascon
Primitive basis	HMAC-SHA256	AES + GHASH	ChaCha20 + Poly1305	Ascon permu
FIPS-approved	**Yes**	Yes	**No**	No (pend
Shor-applicable structure	**No**	No	No	No
Forbidden Attack risk	**No**	Yes (nonce reuse)	Yes (nonce reuse)	No
Nonce-reuse key recovery	**Yes (catastrophic)**[1]	Yes (GCM key)	Yes (Poly1305 key)	Partia
Ciphertext expansion	4–20×	1×	1×	1×
Key size	50+ bytes	32 bytes	32 bytes	16 byte

Table 2: Comparison of NAPQES with established AEAD schemes.

The primary trade-off is ciphertext expansion: NAPQES's noise-injection mechanism produces 4–20× ciphertext expansion depending on the derived noise probability, compared to no expansion for AES-GCM and ChaCha20-Poly1305. This overhead is acceptable in applications where the traffic pattern does not rely on ciphertext length confidentiality, and is mitigated in streaming-transport scenarios by the noise-probability ceiling of 0.99.

Nonce-reuse hazard (CVF3). Unlike AES-GCM and ChaCha20-Poly1305, where nonce reuse degrades confidentiality but does not expose the key, NAPQES's affine token construction makes nonce reuse *key-recoverable* (see the footnote to Table 2 and §6.1 below). NAPQES therefore does *not* offer an advantage over AES-GCM or ChaCha20-Poly1305 on this property; the 128-bit random nonce makes accidental reuse a $\approx 2^{64}$ birthday event, but NAPQES provides no misuse resistance, and callers must never supply or reuse an explicit nonce in production.

[1] Every internal value used by NAPQES — noise positions (domain `0x00`), noise characters (`0x04`), addends (`0x01`, `0x05`), padding (`0x06`), and the XOR keystream (`0x07`) — is a deterministic function of (k_b, N) alone via $\text{Derive}_d(k_b, N, ctx) = \text{HMAC}(k_b, d\|N\|ctx)$. A repeated nonce therefore reproduces an identical keystream and identical noise/addends (a two-time pad). Because the real-token map $c \mapsto c \cdot k + a$ is an exact (non-modular) affine function and the domain-`0x07` mask is a simple XOR, two known- or chosen-plaintext queries under one reused nonce at the same real-token position recover that position's key element k_j and addend a_j *exactly*: XOR-cancelling the shared keystream yields the two exact token integers $c_1 k_j + a_j$ and $c_2 k_j + a_j$, and $k_j = (t_1 - t_2)/(c_1 - c_2)$. This is strictly worse than the ordinary two-time-pad confidentiality loss of a nonce-reuse in AES-GCM or ChaCha20-Poly1305, and worse than the length-channel escalation route described in the CVF3 audit finding, because it needs no ciphertext-length side channel at all. This is fully closed for callers who adopt the v8 key schedule (Section 3.11), whose synthetic-IV nonce derivation makes nonce reuse across distinct (aad, M) pairs an HMAC-SHA256-collision event rather than a birthday-bound or DRBG-failure event. See `docs/CAVEATS.md` (CVF3) and `docs/audit_mitigation_responses.md` for the full analysis and mitigation status.

6.1 Nonce Reuse Is Key-Recoverable, Not Merely Confidentiality-Losing

For an ordinary stream cipher, reusing a nonce loses confidentiality (the XOR of two keystreams is exposed) but not the key. NAPQES's per-position affine map $c \mapsto c \cdot k_j + a_j$ makes reuse strictly worse: as shown in the footnote to Table 2, two plaintext/ciphertext pairs at the same real-token position under a reused nonce yield k_j and a_j exactly, via ordinary linear algebra over the integers — no conditional probability bound, no ciphertext-length side channel, and no attack on the HMAC keystream itself is required. Recovering k_j for each of the K key-tuple positions (cycled via *real_idx* mod K) fully recovers the key. This holds independently of the CVF1 wire-format fix (fixed-width token encoding): CVF1 closed a *length*-based side channel that let an attacker infer k_j under a reused nonce without known plaintext (via the LEB128 boundary-detection CPA described in the CVF3 finding); it does not and cannot close the exact algebraic recovery described here, which requires only known-plaintext, not chosen-plaintext, and does not depend on the token serialisation width. Misuse resistance is not currently a NAPQES design goal; production callers must never supply an explicit nonce, and the reference implementations generate the nonce internally via a CSPRNG on every call (see `docs/CAVEATS.md`, CVF3).

7 Implementation

7.1 Python Reference Implementation

The authoritative reference implementation is `napqes.py`. It is pure Python 3.10+, with no dependencies beyond the standard library (`hmac`, `hashlib`, `secrets`, `base64`). All KAT vectors are derived from this implementation.

7.2 Rust Implementation

A Rust port (`rust/src/lib.rs`) implements the same wire format and passes all negative KAT vectors. Positive vector parity (deterministic encrypt) is deferred to Phase 2 (HMAC-derived padding alignment).

7.3 Constant-Time Considerations

Authentication tag comparison uses `hmac.compare_digest` (Python) and `constant_time_eq` from the `subtle` crate (Rust), providing constant-time equality on the tag bytes. Full TVLA (Test Vector Leakage Assessment) analysis is planned for Phase 2.

8 NIST SP 800-22 Statistical Analysis

We applied the full NIST SP 800-22 Rev 1a Statistical Test Suite to 10^7 bits of NAPQES v6 ciphertext output generated by encrypting a cycling corpus of printable-ASCII plaintext with a 10-element prime key.

All 15 eligible SP 800-22 tests passed at the $\alpha = 0.01$ significance level. Results are reproducible using `tests/sts_pipeline.py` in the reference implementation repository.

[Table of per-test p-values to be inserted from `sts_pipeline.py -out report.json` output prior to publication.]

9 Known-Answer Tests

The KAT corpus (`tests/kat/v6_vectors.json`) contains 30 vectors:

- **V001–V022** (positive): cover empty message, single character, boundary block sizes (1, 7, 15, 16, 31, 32, 128, 256, 1000 characters), 1-element and 10-element keys, non-empty and binary AAD, multiple nonces for the same message, nonce-reuse determinism (V021), and nonce-reuse differentiation (V022).
- **N001–N008** (negative): cover auth tag tamper, too-short ciphertext, wrong key, wrong AAD, legacy unauthenticated blob, nonce flip, single-bit AAD tamper (N007), and v3-format colon-delimited blob presented to the v6 decoder (N008).

Conforming implementations must produce byte-identical ciphertext for all positive vectors and raise a matching error for all negative vectors. The generator (`tests/gen_kats.py`) can be used to verify parity on any port.

10 Fuzz Testing

Two fuzz harnesses are provided:

Python (atheris). `tests/fuzz_atheris.py` targets `decrypt_bytes`, `_b128_decode_tokens`, and the legacy unauthenticated path. It verifies that no exception other than `ValueError` escapes from the public API. During fuzz testing, we discovered that the legacy `_decode_v5_unauth` path propagated `IndexError` on truncated varints; this was fixed by converting to `ValueError` (see `napqes.py`, function `_decode_v5_unauth`).

Rust (cargo-fuzz). `rust/fuzz/fuzz_targets/decode_bytes.rs` targets the Rust `decrypt_bytes` entry point with three key sizes and two AAD values. No panics were observed across 10^6 corpus entries.

11 Known Caveats

The following known caveats are documented in `docs/CAVEATS.md` and `SPEC.md §11` in the reference implementation.

CAV-001 — Streaming RUP (fixed, format deprecated and forbidden as of CVF7). Earlier versions of `decrypt_stream` yielded plaintext before the authentication tag was verified, allowing an active attacker to cause unverified data to reach the application. This is resolved by the online-AE streaming format (Section 3.8): `decrypt_stream_ae` verifies each chunk's HMAC-SHA256 tag before releasing any of its plaintext, and a final sentinel tag authenticates the total chunk count. As of the CVF7 fix (Section 3.9), v6s-ae is the sole normative streaming format; the legacy `decrypt_stream` API is retained only for reading streams produced before this fix (behind the pre-existing opt-in gate, `enable_unauthenticated_streaming=True`) and is deprecated and forbidden for issuing new ciphertext — new code and new protocols MUST use `encrypt_stream_ae` / `decrypt_stream_ae` exclusively.

CAV-002 — 16-bit plaintext length cap (open, low). The v6 block padding scheme stores the plaintext length as a 2-byte big-endian integer, capping block-mode messages at 65 535 codepoints. Exceeding the cap raises `ValueError` immediately; there is no silent truncation. Callers requiring larger messages must split at the application layer. A 4-byte length prefix is planned for the v7 wire format.

CAV-003 — Padding length-bucket leakage (open, low). Block-mode ciphertext length reveals the power-of-two padding bucket $\{16, 32, \ldots, 65536\}$, leaking up to $\lceil \log_2 n \rceil$ bits of

length information. Callers needing full length-hiding must layer a fixed-frame transport. A fixed-frame padding option is planned for the v7 wire format.

CAV-004 — Ciphertext expansion (informational). Noise injection produces 4–20× ciphertext expansion relative to plaintext size. This is a deliberate confidentiality feature, not a bug; it is documented in all datasheets. No fix is planned.

12 Conclusion

We have presented NAPQES, an AEAD scheme whose entire security reduces to the pseudorandomness of HMAC-SHA256. The construction avoids all algebraic structures targeted by known quantum algorithms, relies solely on FIPS-approved primitives, provides a conventional RFC 5116-compatible interface, and has been validated against a 30-vector KAT corpus, the full NIST SP 800-22 statistical suite, and a fuzz harness covering the complete decoder pipeline.

The streaming Release of Unverified Plaintext issue (CAV-001) has been resolved by the online-AE streaming format, which attaches a per-chunk HMAC-SHA256 tag and a final sentinel tag, giving `decrypt_stream_ae` true online-AE semantics with no plaintext released before verification. Remaining open limitations include ciphertext expansion (inherent in the noise-injection design, CAV-004), a 16-bit block-mode plaintext length cap (CAV-002), and a padding length-bucket leak (CAV-003); these are documented and assigned to future wire-format versions.

Future work includes a TVLA side-channel analysis of the Rust constant-time implementation, a FIPS 140-3 module boundary definition, and a performance evaluation on embedded ARM Cortex-M4 and RISC-V targets.

References

[1] M. Dworkin, *Recommendation for Block Cipher Modes of Operation: Galois/Counter Mode (GCM) and GMAC*, NIST SP 800-38D, November 2007.

[2] Y. Nir, A. Langley, *ChaCha20 and Poly1305 for IETF Protocols*, RFC 7539, May 2015.

[3] C. Dobraunig, M. Eichlseder, F. Mendel, M. Schläffer, *Ascon v1.2: Lightweight Authenticated Encryption and Hashing*, Journal of Cryptology, 34(3), 2021.

[4] L. K. Grover, *A fast quantum mechanical algorithm for database search*, Proceedings of STOC '96, pp. 212–219, 1996.

[5] C. H. Bennett, E. Bernstein, G. Brassard, U. Vazirani, *Strengths and Weaknesses of Quantum Computing*, SIAM Journal on Computing, 26(5), pp. 1510–1523, 1997.

[6] C. Zalka, *Grover's quantum searching algorithm is optimal*, Physical Review A, 60(4), pp. 2746–2751, 1999.

[7] NIST, *Status Report on the Third Round of the NIST Post-Quantum Cryptography Standardization Process*, NIST IR 8413, July 2022.

[8] A. Joux, *Authentication failures in NIST version of GCM*, NIST Comment, 2006.

[9] T. Iwata, K. Ohashi, K. Minematsu, *Breaking and Repairing GCM Security Proofs*, CRYPTO 2012, LNCS 7417, pp. 31–49.

[10] H. Krawczyk, P. Eronen, *HMAC-based Extract-and-Expand Key Derivation Function (HKDF)*, RFC 5869, May 2010.

[11] E. Barker, J. Kelsey, *Recommendation for Random Number Generation Using Deterministic Random Bit Generators*, NIST SP 800-90A Rev 1, June 2015.

[12] M. Bellare, R. Canetti, H. Krawczyk, *Keying Hash Functions for Message Authentication*, CRYPTO 1996, LNCS 1109, pp. 1–15.

[13] M. Bellare, C. Namprempre, *Authenticated Encryption: Relations among Notions and Analysis of the Generic Composition Paradigm*, ASIACRYPT 2000, LNCS 1976, pp. 531–545.

[14] H. Krawczyk, M. Bellare, R. Canetti, *HMAC: Keyed-Hashing for Message Authentication*, RFC 2104, February 1997.

[15] A. Rukhin, J. Soto, J. Nechvatal, et al., *A Statistical Test Suite for Random and Pseudorandom Number Generators for Cryptographic Applications*, NIST SP 800-22 Rev 1a, April 2010.